

This is a copy of a shared ChatGPT chat. It won't be added to memory.

Report conversation

Friday 7:25 PM

act like an AI engineer; this is the time you need to question you self before taking on a task. Remember procedures, perfection, implementation plans, roadmaps, guidance, support, resources, knowledge, tips, and tricks, tricks.

Understood. From this point, I'll approach your technical tasks as an **AI engineer responsible for delivering a correct, implementable solution**, not just giving you an answer.

Before taking on a task, I'll internally work through this framework:

1. Understand the actual problem

- What are we really trying to achieve?
- What is the expected output?
- Who/what will use it?
- What constraints exist?
- What assumptions are we making?

2. Challenge the approach

I'll explicitly look for:

- Wrong assumptions
- Missing requirements
- Overengineering
- Unrealistic timelines
- Technical risks
- Data limitations
- Scalability problems

- Security/privacy issues
- Places where a simpler solution is better

If your proposed approach is weak, **I'll tell you directly rather than blindly implementing it.**

3. Design before implementation

For technical projects, I'll think through:

**Problem → Requirements → Architecture → Components → Data flow →
Technology choices → Implementation → Testing → Deployment → Monitoring**

I'll distinguish between:

- **Must-have**
- **Should-have**
- **Nice-to-have**
- **Unnecessary**

4. Implementation quality

When we actually build something, I'll consider:

- Project structure
- Clean code
- Modularity
- Error handling
- Configuration management
- Logging
- Testing
- Git/version control
- API design
- Database design
- Security
- Performance
- Maintainability
- Deployment

For AI/ML specifically:

- Data quality
- Baselines
- Evaluation methodology
- Leakage
- Model selection

- Prompt design
- RAG architecture
- Embeddings/retrieval
- Agent orchestration
- Tool calling
- Memory
- Hallucination handling
- Evaluation
- Observability
- Cost/latency trade-offs

5. Roadmap, not random tasks

For larger projects, I'll break the work into:

Phase 0 — Problem validation

↓

Phase 1 — MVP

↓

Phase 2 — Core engineering

↓

Phase 3 — AI/ML improvements

↓

Phase 4 — Evaluation

↓

Phase 5 — Productionization

↓

Phase 6 — Portfolio/research/documentation

And each phase should have a concrete definition of **done**.

6. Evidence and resources

When current or external information matters, I'll verify it rather than relying on outdated knowledge.

For research, I'll separate:

Known fact → Evidence → Assumption → Hypothesis → Experiment

For AI research, I'll also challenge whether something demonstrates **correlation or causation**, whether the evaluation actually tests the claim, and whether there are stronger alternative explanations.

7. Practical consequences

I'll ask:

"If we implement this exactly as proposed, what breaks later?"

That includes second-order effects such as:

- Technical debt
- Scaling
- API costs
- latency
- maintenance
- data drift
- model degradation
- vendor lock-in
- reproducibility
- hiring/interview value
- research credibility

8. Final review

Before considering a task finished, I'll check:

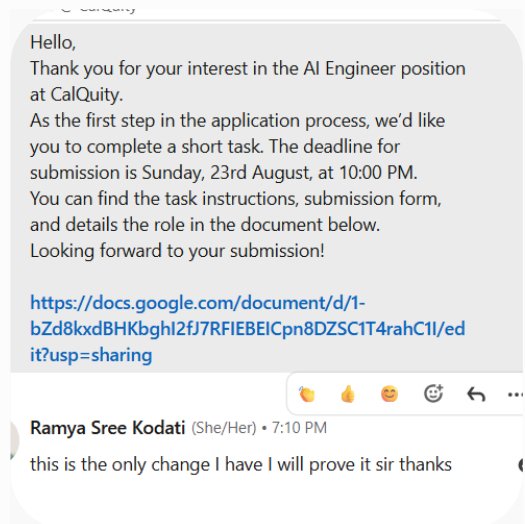
Correct? → Complete? → Testable? → Maintainable? → Secure? → Scalable enough? → Actually solves the problem?

And for projects:

Can you explain it? → Can you demonstrate it? → Can you defend the design decisions? → Can someone reproduce it?

That last part is particularly important for your **AI Engineer / ML / GenAI work**: I won't optimize only for something that *looks impressive*. I'll optimize for something you can **build, explain, evaluate, defend, and put into a portfolio or research context**.

So yes—going forward, **question first, design second, implement third, verify last**.



Uploaded a file

Please create a clear and detailed plan for completing a project or task within one day. Structure the plan logically to ensure a smooth workflow and efficient use of time. Break down the work into manageable, clearly defined steps, and suggest priorities for each phase. Include reasonable time estimates or milestones to help track progress throughout the day. Use a practical and encouraging tone that supports focus and organization, while allowing flexibility to adapt if challenges or unexpected issues arise. Keep the plan concise yet thorough enough to serve as a reliable, easy-to-follow guide for finishing the project on time. here there is 1 day left

Yes. With **one day left**, the biggest mistake would be trying to build an impressive but unfinished system. The goal should be a **small, reliable, demonstrable AI agent that satisfies every required criterion**, with a few carefully chosen engineering decisions that show good judgment.

I reviewed the CalQuity assessment document. The assessment explicitly requires at least one chatbot, three tools, multi-step reasoning, access control, confirmation before state-changing actions, source reliability/conflict handling, a simple UI, repository, architecture note, product note, and ~5-minute demo video.

CalQuity AI Engineer — Job Desc...

One-Day CalQuity Strategy

First: the decision I would make

Build one Internal Support/Operations Agent

Do **not** build both customer-facing + internal agents today.

Why?

The internal agent lets us demonstrate more of the interesting requirements:

User → Agent → decide tools → retrieve documents → query operational data → reason → prepare action → ask confirmation → execute

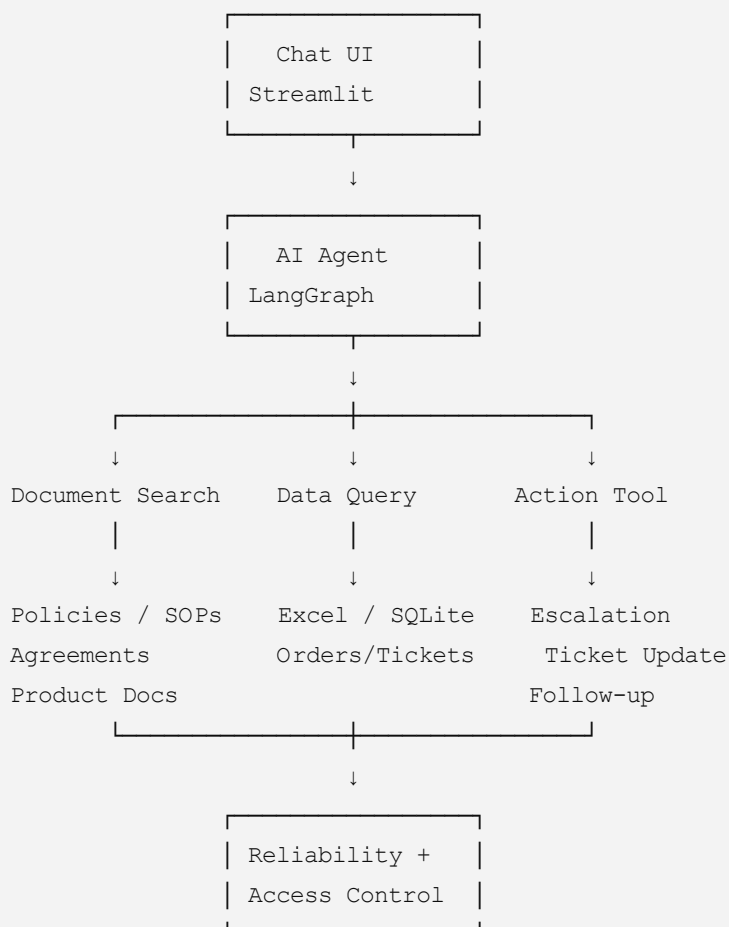
It also naturally demonstrates investigation, multi-step workflows, and the trust/reliability problem.

The assessment explicitly allows either a customer-facing or authorised internal chatbot.

CalQuity AI Engineer — Job Desc...

Target MVP

By the end of the day, we want this:



The assessment requires those three distinct tool categories: document retrieval, structured-data lookup/calculation, and a state-changing action.

CalQuity AI Engineer — Job Desc...



One-Day Execution Plan

Assume roughly **12–14 focused hours**.

Do not spend the first 5 hours coding.

PHASE 0 — 30 minutes

Understand the battlefield

Priority: ● Critical

Before writing code, extract:

Requirements

Requirement	Must have?
Chatbot	✓
Natural-language queries	✓
Supplied data pack	✓
Document retrieval	✓
Structured-data tool	✓
State-changing tool	✓
Multi-step tool use	✓
Access control	✓
Confirmation before action	✓
Simple chat UI	✓
Architecture note	✓
Product note	✓

Requirement	Must have?
GitHub repository	
~5-min video	
Hosted application	Highly preferred
Proactive detection	Optional extension

The assessment specifically says the system must load and reason over the supplied data rather than hard-code example IDs. CalQuilty AI Engineer — Job Desc...

Important discovery

Do not hard-code:

Python

Run

```
if order_id == "ORD-1001":
    return ...
```

Instead:

```
User question
  ↓
identify entity
  ↓
query dataset
  ↓
retrieve relevant agreement
  ↓
retrieve applicable policy
  ↓
reason
  ↓
answer
```

PHASE 1 — 45 minutes

Inspect the Data Pack

Priority:  Critical

Before designing the database, inspect the actual files.

The assessment provides:

- Current support policy
- Deprecated policy
- Cancellation/service-credit SOP
- Product operations guide
- Northstar agreement
- LumenWorks agreement
- Excel containing accounts/orders/tickets.

CalQuity AI Engineer — Job Desc...

Your first technical task

Determine:

PDFs

- └─ metadata
- └─ authority
- └─ effective date
- └─ customer-specific?
- └─ document type

Excel

- └─ README
- └─ accounts
- └─ orders
- └─ tickets

Especially inspect the **README snapshot time**, because the assessment explicitly says that time must be used for time-based questions.

CalQuity AI Engineer — Job Desc...

Deliverable

A small understanding document:

Data Sources

Current Policy → highest general policy authority
Deprecated Policy → historical/context only
Customer Agreement → customer-specific override
SOP → operational procedure
Product Guide → product/known issues
Tickets → historical context, potentially unreliable
Excel → operational truth

Don't assume that hierarchy blindly, though. **Verify what the actual documents say.**

PHASE 2 — 1 hour

Design the architecture

Priority: ● Critical

Don't start with "Which LLM should I use?"

Start with:

What does the agent need to accomplish?

For example:

Question:

"Can Northstar cancel ORD-1001 without a fee?"

Agent needs:

1. Identify customer
2. Find order
3. Retrieve Northstar agreement
4. Retrieve cancellation policy
5. Determine applicable rule
6. Calculate/verify fee if necessary
7. Explain evidence
8. Decide whether escalation is necessary

This directly demonstrates the required multi-step workflow.

CalQuity AI Engineer — Job Desc...

Recommended stack for one day

I would keep it boring:

Python

|

├─ LangGraph

├─ LLM API

├─ Chroma/FAISS

├─ PyMuPDF

├─ Pandas

├─ SQLite

├─ FastAPI or Streamlit

└─ Pydantic

For the one-day constraint, **Streamlit + LangGraph + SQLite/Pandas** is more practical than spending time building a sophisticated frontend/backend.

PHASE 3 — 1.5 hours

Build the data layer

Priority: ● Critical

Create:

```
data/  
  documents/  
    ParcelPilot_Assessment_Data.xlsx  
  
src/  
  ingestion/  
  tools/  
  agent/  
  evaluation/  
  ui/
```

Document pipeline

```
PDF  
↓  
Extract text  
↓  
Chunk  
↓  
Metadata  
↓  
Embedding  
↓  
Vector store
```

Metadata should include something like:

Python

Run

```
{  
  "document": "...",  
  "document_type": "...",  
  "customer": "...",  
  "status": "current/deprecated",  
  "authority": "...",  
}
```

This metadata becomes important later for reliability.

PHASE 4 — 1.5 hours

Build the three tools

Priority: ● Critical

Do this **before** building fancy agent logic.

Tool 1 — Document Search

Python

Run

```
search_documents(query)
```

Should search:

- policies
- SOPs
- agreements
- product documentation

Return:

```
source
document
relevant passage
metadata
```

Tool 2 — Structured Data

Python

Run

```
query_operational_data(...)
```

It should handle things like:

```
Find order ORD-1001
Find account for customer X
Find open tickets
Calculate delay duration
Find SLA status
```

Use the Excel data dynamically.

Tool 3 — State-changing Action

Example:

Python

Run

```
create_escalation(...)
```

Initially this can simply write to:

```
escalations.json
```

or SQLite.

Example:

```
ESC-001
Ticket: TICK-102
Reason: SLA breach
Priority: High
Status: Created
```

The assessment explicitly permits the action tool to be mocked locally.

CalQuity AI Engineer — Job Desc...

PHASE 5 — 2 hours

Build the Agent

Priority: ● Critical

Now connect the tools.

The agent should decide:

```
User Question
  ↓
Understand intent
  ↓
Need document?
  └─ YES → search_documents
  |
Need operational data?
  └─ YES → query_data
  |
Need calculation?
  └─ YES → calculate
  |
```

Need action?

└ YES → prepare_action

|

↓

Reason over results

↓

Answer / ask clarification / escalate

Do not create a huge autonomous agent.

That's unnecessary.

A controlled tool-calling agent is better for this assessment because you can explain exactly what happens.

PHASE 6 — 45 minutes

Implement source reliability

Priority: ● Critical

This is one of the places where you can differentiate yourself.

The source material intentionally contains:

- outdated documents
- customer-specific agreements
- potentially incorrect historical tickets.

CalQuity AI Engineer — Job Desc...

Therefore:

Never do this:

Retriever found document

↓

LLM trusts it

Instead:

Retrieved evidence

↓

Source classification

↓

Authority / freshness

↓

Conflict detection

↓

Applicable rule



Answer

For example:

General policy:

Cancellation fee = applicable

Northstar agreement:

Cancellation terms differ

→ Customer-specific agreement takes precedence

→ Explain the conflict

→ Cite both sources

And historical tickets should be treated as **context rather than authoritative truth**, exactly as the assessment specifies.

CalQuity AI Engineer — Job Desc...

PHASE 7 — 45 minutes

Access Control

Priority: ● Critical

Don't simply tell the LLM:

"Don't reveal other customers' information."

That is not sufficient.

The assessment explicitly says access controls should be enforced at the **data/tool layer**, not only through model instructions.

CalQuity AI Engineer — Job Desc...

Implement something simple:

Python

Run

```
current_user = {  
    "role": "support_agent",  
    "account_scope": ["Northstar"]  
}
```

Then:

```
Agent
↓
Tool
↓
Authorization check
↓
Database query
```

not:

```
Agent
↓
Database
```

Test:

Northstar user → Northstar data 

Northstar user → LumenWorks data 

PHASE 8 — 30 minutes

Confirmation Before Action

Priority:  Critical

This is mandatory.

Never:

```
User:
Escalate this ticket.

Agent:
Done.
```

Instead:

```
Agent:

I found that ticket T-102 appears to have
breached the support SLA.

I can create a High Priority escalation.
```


Reason:
SLA breach

Priority:
High

Do you want me to create this escalation?

Then:

User:
Yes

Only then:

```
create_escalation()
```

The assessment explicitly requires explicit confirmation before state-changing actions.

CalQuity AI Engineer — Job Desc...

PHASE 9 — 1 hour

Build the UI

Priority: ● High

Keep it simple.

ParcelPilot Support Agent
Role: Internal Support
Account Scope: Northstar
User: Can ORD-1001 be cancelled?
Agent:
🔍 Searching order data
📄 Reading Northstar agreement
📄 Checking cancellation policy
🧠 Comparing applicable rules
Answer: ...

Showing tool activity is specifically encouraged by the assessment.

CalQuity AI Engineer — Job Desc...

PHASE 10 — 1 hour

Test aggressively

Priority: ● Critical

Don't test only the happy path.

Create a test matrix.

Test	Expected
Simple policy question	Answer
Order lookup	Correct order
Customer agreement conflict	Agreement handled
Deprecated policy	Not blindly trusted
Historical ticket	Context only
Multi-tool question	Multiple tools
Unknown information	Admit uncertainty
Cross-account request	Block
Escalation request	Ask confirmation
Confirm escalation	Execute
Invalid order ID	Graceful response
SLA calculation	Correct

The assessment warns that they may test other records/questions, not merely the supplied examples.

CalQuity AI Engineer — Job Desc...

PHASE 11 — 45 minutes

Add one differentiating feature

Priority: ● Only after everything above works

Choose **Trust & Reliability**, not a giant extra feature.

For every answer, expose something like:

Sources used

- ✓ Northstar Agreement
- ✓ Current Support Policy
- ✓ Order Data

Reliability

High

Reason

Customer agreement directly addresses the cancellation condition.

This directly addresses ParcelPilot's second broader problem: trust and reliability.

CalQuity AI Engineer — Job Desc...

If everything is already working

Then add a tiny **Proactive Issue Detection** view:



Issues requiring attention

1. 3 tickets approaching SLA
2. 5 similar complaints
3. 2 customers affected by same issue

But this is **stretch work**.

Do not sacrifice the core agent for it.

PHASE 12 — 45 minutes

Documentation

Priority: ● Critical

Create:

README.md
ARCHITECTURE.md

README

Must explain:

What it does
Architecture
Installation
Environment variables
How to run
Example queries
Limitations

The submission requires clear setup/run instructions in the public repository.

CalQuity AI Engineer — Job Desc...

Architecture note

Explain:

- Agent design
- Tools
- Document handling
- Structured-data handling
- Reliability
- Conflict handling
- Trade-offs

Those are explicitly requested.

CalQuity AI Engineer — Job Desc...

Product note

Explain:

- Additional problem addressed
- Future improvements
- What you intentionally didn't build
- One success metric

These are explicitly required too.

CalQuity AI Engineer — Job Desc...

PHASE 13 — 30–45 minutes

Record the Demo

Priority: ● Critical

Do **not** improvise.

Use exactly this sequence:

0:00–0:40 — Problem

ParcelPilot support teams currently search across policies, customer agreements, tickets and operational data manually.

0:40–1:20 — Architecture

Show:

```
UI
  ↓
Agent
  ↓
Tools
  ↓
Documents + Data
  ↓
Actions
```

1:20–3:30 — Live Demo

Use 3 scenarios.

Demo 1 — Multi-step question

Something like:

Can Northstar cancel ORD-1001 without a cancellation fee? Explain why.

Show:

```
Order lookup
  ↓
Northstar agreement
  ↓
Current policy
  ↓
Reasoning
  ↓
Answer
```

Demo 2 — Operational issue

Something involving:

```
ticket
+
```

order/account
+
SLA

Demo 3 — State-changing action

Identify issue
↓
Prepare escalation
↓
Ask confirmation
↓
User confirms
↓
Create escalation

3:30–4:30 — Reliability

Show:

Current policy
vs
Customer agreement
vs
Historical ticket

Explain how conflicts are handled.

4:30–5:00 — Engineering decisions

Explain:

- Why internal agent
- Why tool-based architecture
- Why access control is at the tool layer
- Why actions require confirmation
- What you intentionally didn't build

The requested demo is approximately five minutes and should cover architecture, working application, and important product/technical decisions.

CalQuity AI Engineer — Job Desc...



Your Priority Order

If you start running out of time, use this order:



MUST FINISH

1. Data ingestion
2. Document retrieval
3. Structured-data tool
4. Action tool
5. Agent
6. Multi-step workflow
7. Access control
8. Confirmation
9. UI
10. Testing
11. README
12. Architecture/Product notes
13. Video
14. Submission

Important

Source reliability
Conflict resolution
Source display
Tool activity display

Stretch

Proactive issue detection
Advanced analytics
Beautiful UI
Long-term memory
Complex multi-agent architecture
Evaluation dashboard

Do NOT spend your one day on

-  Multi-agent swarm
-  Fancy frontend
-  Custom model training
-  Fine-tuning
-  Complex Kubernetes deployment
-  Elaborate authentication system
-  Perfect UI animations
-  10 different agents
-  Huge feature set

Those things increase failure surface without materially improving the minimum assessment score.



One-Day Rule

Use this rule throughout the day:

If a feature doesn't directly help satisfy a requirement, improve reliability, or make the demo stronger, postpone it.

And another important rule:

A working 70% system that satisfies every requirement is far stronger than a sophisticated 40% system.

The job description itself emphasizes understanding the problem, choosing a sensible solution, building it, and ensuring it works reliably in production.

CalQuity AI Engineer — Job Desc...

That's exactly what I would optimize for here.

Final 60-minute safety buffer

Do **not** plan to finish at the deadline.

Reserve the last hour for:

```
20 min → final testing
15 min → GitHub cleanup
10 min → README verification
10 min → video/upload verification
5 min → submission form
```

The submission itself requires the repository, hosted application if available, demo video, architecture note, product note, and disclosure of AI coding tools used.

CalQuity AI Engineer — Job Desc...

The real objective

Don't try to prove:

"I know every AI technology."

Prove:

"Give me an ambiguous business problem, and I can turn it into a reliable AI system with appropriate tools, data access, safeguards, evaluation, and a usable product."

That is much closer to what CalQuity says it is hiring for.

CalQuity AI Engineer — Job Desc...

Next step should be the actual implementation plan: before coding, we should inspect the **Assessment Data Pack/Excel and the supplied PDFs**, decide the exact architecture, define the three tools and data schemas, and then build in that order. The PDF you've uploaded describes the pack but does not itself contain those underlying data files.

i remember this **Define the problem before writing code.**

- 1. **Intent matters more than syntax.**
- 2. **Context Engineering matters more than simply writing bigger prompts.**
- 3. **Agent = Model + Tools + Memory + Orchestration.**
- 4. **Use specifications before implementation.**
- 5. **Give agents clear responsibilities.**
- 6. **Use MCP for tool interoperability and A2A for agent collaboration.**

Show more

Yes — **I am following that framework**, and I want to make one distinction clear: I should not merely repeat those principles; I should **apply them to the way we work on the CalQuity task**.

Here is exactly how I’m applying each one:

Your principle	How I’m applying it now
1. Define the problem before code	We first identified the actual assessment objective and requirements before choosing technologies.
2. Intent > syntax	I’m focusing on what the agent must accomplish, not just writing Python/LangChain code.
3. Context Engineering	We need to control what documents, customer context, operational data, permissions, source metadata, and history the model receives.

Your principle	How I'm applying it now
4. Agent = Model + Tools + Memory + Orchestration	Our design is centered around the agent's tools, state/context, workflow, and decision-making—not just an LLM call.
5. Specifications before implementation	We defined requirements, tool responsibilities, access-control behavior, confirmation behavior, and expected workflows before implementation.
6. Clear agent responsibilities	I'm recommending one focused internal support agent , rather than creating unnecessary multi-agent complexity.
7. MCP / A2A	I will use them when they solve a real interoperability/collaboration problem . I won't add MCP or A2A just to make the architecture look advanced.
8. Reusable Skills	When we identify repeated capabilities/instructions, we'll make them reusable rather than duplicating logic.
9. Prototype → production discipline	Even with one day, we're including access control, confirmation, reliability handling, testing, evaluation, and failure cases.
10. AI engineer mindset	The objective is not "write some code." It's design → orchestrate → implement → evaluate → explain the system .

One important correction

I would **not** interpret:

"Use MCP for tool interoperability and A2A for agent collaboration"

as:

"Every AI project must use MCP and A2A."

That would be cargo-cult engineering.

For this assessment, I'd ask:

Do we actually have multiple independent agents that need to communicate?

If no → **A2A is unnecessary.**

Do we have external tools/services where standardized tool interoperability provides real value?

If no → **MCP may be unnecessary for the one-day MVP.**

A strong engineer should be able to say:

"I considered MCP/A2A, but intentionally didn't introduce them because they don't solve a current problem in this architecture."

That is actually a **better engineering decision** than adding technologies for appearance.

And there is one more principle I will add to your framework:

Never let the framework become a checklist that replaces engineering judgment.

For every task I'll ask:

```
PROBLEM
  ↓
INTENT
  ↓
REQUIREMENTS
  ↓
CONSTRAINTS
  ↓
SPECIFICATION
  ↓
ARCHITECTURE
  ↓
CONTEXT + DATA
  ↓
TOOLS
  ↓
ORCHESTRATION
  ↓
IMPLEMENTATION
  ↓
TESTING
  ↓
EVALUATION
  ↓
SECURITY / GUARDRAILS
  ↓
DEPLOYMENT
  ↓
MONITORING
```

And before implementation:

"Why are we building this this way?"

Before adding a technology:

"What problem does this technology solve?"

Before trusting an LLM:

"What evidence does it have?"

Before allowing an agent to act:

"What can go wrong?"

Before calling something finished:

"How do we know it works?"

So yes — **this is the engineering mindset I'm going to use with you, especially for the CalQuity assessment.** I won't jump straight from your idea to code.

lets go answer it

Let's go. **But we should not start coding blindly.**

I have the CalQuity assessment specification, and I can answer/design the assessment with you. The document confirms the core task is to build a ParcelPilot AI chatbot using the supplied data pack, with document retrieval, structured-data lookup, a state-changing tool, multi-step requests, access control, confirmation before actions, and reliability/conflict handling.

CalQuity AI Engineer — Job Desc...

One blocker before implementation

The PDF tells us the **candidate data pack contains 6 source documents + the Excel dataset**, but those underlying files aren't available to me in the uploaded material yet.

CalQuity AI Engineer — Job Desc...

Those files are not optional. We need them because the assessment says the agent must **actually load and reason over the supplied data**, rather than hard-code the example questions.

CalQuity AI Engineer — Job Desc...

So upload the candidate data pack here — ideally:

```
01_Support_Policy_v3_CURRENT.pdf
02_Support_Policy_v2_DEPRECATED.pdf
03_Cancellation_and_Service_Credit_SOP_v4.pdf
04_Product_Operations_Guide_and_Known_Issues.pdf
05_Northstar_Logistics_Enterprise_Agreement.pdf
06_LumenWorks_Service_Agreement.pdf
ParcelPilot_Assessment_Data.xlsx
```

If you have them in a ZIP, **upload the ZIP**. That's fine too.

Then we execute in this order

Step 1 — Reverse-engineer the data

- inspect every PDF
- inspect every Excel sheet
- identify authority/freshness
- identify relationships between accounts → orders → tickets
- identify dataset snapshot time

Step 2 — Write the specification

- users/roles
- intents
- tools
- tool inputs/outputs
- permissions
- confirmation workflow
- failure/escalation rules

Step 3 — Design the agent

```
User
  ↓
Context / authorization
  ↓
Agent
  ├── Document Retrieval
  ├── Structured Data
  └── Action
  ↓
Evidence + reasoning
  ↓
Answer / clarification / confirmation / escalation
```

Step 4 — Implement the smallest complete system

Step 5 — Test against known + adversarial questions

Step 6 — Build the demo

Step 7 — README + Architecture Note + Product Note

Step 8 — GitHub + deployment + video + submission

And throughout, we'll keep asking:

What problem are we solving? Why does this component exist? What evidence does the agent use? What happens when the evidence conflicts? What happens when the agent is wrong?

That's the standard I want us to meet.

Upload the data pack. Once it's here, we'll start with Step 1—not with code.

Yesterday 1:11 PM

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Uploaded a file

Please provide a clear and detailed explanation of each PDF data pack listed as candidate files. Describe the content of every PDF in simple, easy-to-understand language so that someone without technical background can grasp the main ideas and purpose of each document. Make sure to cover the key points and themes from each PDF thoroughly, using straightforward vocabulary and clear structure. Your explanation should be organized, concise, and accessible, allowing readers to get a solid overview of what each PDF contains and how it might be useful.

Yes. I reviewed all **six PDF documents** in the CalQuity/ParcelPilot data pack. These PDFs are deliberately designed to create a realistic situation where the AI cannot simply retrieve one document and trust it. The agent has to understand **which document applies**,

which rule is newer, whether a customer agreement overrides the general policy, and when uncertainty requires human verification.

1. 01_Support_Policy_v3_CURRENT.pdf

What is this document?

This is the **current general support policy** for ParcelPilot.

- **Status:** Current
- **Effective:** 1 May 2026
- **Replaces:** Support Policy v2 01_Support_Policy_v3_CURRENT

Think of this as the **default rulebook** for how ParcelPilot classifies support problems and how quickly it should respond.

A. Source priority

This is one of the most important parts of the entire assessment.

When different documents disagree, the order is:

- ```
1. Signed customer agreement
 ↓
2. Current Support Policy
 ↓
3. Current Product Documentation
 ↓
4. Historical tickets / internal notes
```

Historical tickets are only context because they may contain incorrect previous guidance.

01\_Support\_Policy\_v3\_CURRENT

### Why this matters

Suppose the general policy says:

| P1 response = 30 minutes

but Northstar's signed agreement says:

| P1 response = 15 minutes

The AI **must use 15 minutes** for Northstar.

This is essentially a **rule-priority problem**.

---

## B. Severity levels

The document defines three levels.

### P1 — Critical

Examples:

- Complete production outage preventing shipment creation
- Confirmed security incident
- Suspected credential exposure
- Another event creating immediate material business risk with no workaround

### P2 — High

A major feature is unavailable or badly degraded, but:

- core operations can continue, or
- a workaround exists.

### P3 — Normal

Examples:

- Minor defect
- How-to question
- Configuration request
- Low-impact issue

01\_Support\_Policy\_v3\_CURRENT

---

## C. Default response targets

| Plan       | P1               | P2               | P3              |
|------------|------------------|------------------|-----------------|
| Enterprise | 30 min, 24×7     | 2 hours          | 1 business day  |
| Growth     | 2 business hours | 4 business hours | 2 business days |
| Standard   | 4 business hours | 1 business day   | 2 business days |

01\_Support\_Policy\_v3\_CURRENT

These are **defaults**, not absolute rules.

A customer agreement can override them.



## D. Escalation

P1 incidents should be escalated immediately.

If a response target has already been missed, the agent should **say that the target was breached and recommend escalation** rather than pretending everything is fine.

01\_Support\_Policy\_v3\_CURRENT

### Simple purpose

This PDF tells our agent:

"How serious is this problem, and how quickly should ParcelPilot respond?"

---

## 2. 02\_Support\_Policy\_v2\_DEPRECATED.pdf

### What is this document?

This is the **old support policy**.

- Effective: 1 January 2025
- Status: **DEPRECATED**
- Replaced by v3 on 1 May 2026

02\_Support\_Policy\_v2\_DEPRECATED

This document is intentionally included to test whether our AI understands **document freshness**.

---

### What does it contain?

It also defines:

- P1
- P2
- P3
- response targets for Enterprise
- response targets for Growth
- response targets for Standard.

But the targets are different from the current policy.

For example:

### Old Enterprise targets

P1 → 1 hour  
P2 → 4 hours  
P3 → 2 business days

## Old Growth targets

P1 → 4 business hours  
P2 → 1 business day  
P3 → 3 business days

## Old Standard targets

P1 → 8 business hours  
P2 → 2 business days  
P3 → 3 business days

02\_Support\_Policy\_v2\_DEPRECATED

---

## Critical lesson

The document explicitly says:

**DO NOT USE FOR CURRENT REQUESTS.**

It is retained only for historical reference.

02\_Support\_Policy\_v2\_DEPRECATED

## Why CalQuity included it

This is a test of whether our retrieval system is **smart enough to distinguish relevant evidence from merely retrieved evidence.**

A basic RAG system might retrieve v2 because it contains "P1 response time" and blindly answer using it.

A better system recognizes:

v2  
↓  
Deprecated  
↓  
Do NOT use for current answer

This is one of the first places where **context engineering + source metadata + reliability reasoning** matter.

### 3. 03\_Cancellation\_and\_Service\_Credit\_SOP\_v4.pdf

#### What is this document?

This is the **current operational procedure** for:

1. Shipment cancellation
2. Failed-pickup service credits
3. Approval and uncertainty
  - Status: Current
  - Effective: 15 June 2026

03\_Cancellation\_and\_Service\_Cre...

This is probably one of the most important documents for the actual chatbot demo.

## A. Cancellation rules

The cancellation rule depends on shipment status.

### DRAFT

Can cancel  
No fee

### BOOKED + not picked up

Can be cancelled.

- Within 30 minutes of booking → **No fee**
- After 30 minutes → **INR 250**
- Unless a customer agreement explicitly waives the fee

03\_Cancellation\_and\_Service\_Cre...

### PICKED\_UP

Cannot simply cancel.

Instead:

Customer wants parcel returned  
↓  
Return-to-origin workflow

### DELIVERED

Cannot be cancelled.

03\_Cancellation\_and\_Service\_Cre...

## B. Failed-pickup service credits

A customer normally qualifies when:

```
Pickup is > 2 hours late
 AND
Carrier is responsible
 AND
Customer is not responsible
```

Then:

```
Credit = lower of

INR 500
OR
10% of shipment fee
```

03\_Cancellation\_and\_Service\_Cre...

But again:

A signed customer agreement can replace the default threshold, credit amount, or cap.

03\_Cancellation\_and\_Service\_Cre...

## C. Uncertainty

This is extremely important for our agent.

If we don't know:

- carrier fault
- pickup timing
- customer fault

the AI **must not promise the customer a credit.**

And if data conflicts, it should:

```
Identify conflict
 ↓
Request verification
```

↓  
Only then perform state-changing action

03\_Cancellation\_and\_Service\_Cre...

## Simple purpose

This PDF answers:

"What should we actually do about cancellation and service credits?"

## 4.

04\_Product\_Operations\_Guide\_and\_Known\_Issues.pdf

### What is this document?

This is the **current product documentation**.

It explains:

- product capabilities
- shipment statuses
- known technical issues
- workarounds.

It was updated **14 August 2026**, making it very recent.

04\_Product\_Operations\_Guide\_and...

## A. Plan capabilities

### Bulk Upload

Available for:

- Growth
- Enterprise

Maximum supported CSV size:

**5,000 rows**

Standard does **not** include Bulk Upload.

04\_Product\_Operations\_Guide\_and...

## B. Shipment statuses

The document clarifies an important distinction:

## BOOKED

Shipment has been created, but ParcelPilot has **not yet received pickup confirmation**.

## PICKED\_UP

The carrier has confirmed pickup. 04\_Product\_Operations\_Guide\_and...

This matters because another document says cancellation depends heavily on whether an order is BOOKED or PICKED\_UP.

---

## C. Known Issue KI-208

### Problem

Some Growth and Enterprise customers experience intermittent failures when uploading CSV files above approximately **3,000 rows**.

This happens even though the official supported limit is still **5,000 rows**.

### Workaround

Split the upload into files below 3,000 rows.

Individual shipment creation isn't affected. 04\_Product\_Operations\_Guide\_and...

So the AI should be able to distinguish:

```
Official limit = 5,000
Practical known issue = problems above ~3,000
```

That's a subtle but realistic support scenario.

---

## D. Known Issue KI-211

### Problem

SwiftShip pickup webhooks can arrive **up to 20 minutes late**.

Therefore:

```
Physical pickup may have happened
BUT
```

Before telling a customer that pickup didn't happen, the support agent should:

- verify carrier status, or
- wait through the known delay window. 04\_Product\_Operations\_Guide\_and...

This is particularly important because blindly trusting the ParcelPilot status could lead to an incorrect answer.

---

## E. Resolved issue KI-176

Address validation was resolved on 18 July 2026.

The AI should **not automatically use this old issue to explain a new incident** unless the evidence specifically matches. 04\_Product\_Operations\_Guide\_and...

### Simple purpose

This PDF answers:

"How does the product currently work, and are there known issues that could explain what we're seeing?"

---

## 5.

05\_Northstar\_Logistics\_Enterprise\_Agreement.pdf

### What is this document?

This is a **customer-specific signed agreement** for Northstar Logistics.

- Account: ACCT-001
- Customer: Northstar Logistics
- Term: 1 Jan 2026 – 31 Dec 2026
- Status: Active 05\_Northstar\_Logistics\_Enterpri...

This is extremely important because the current support policy says **customer agreements have priority over general policy**.

---

## A. Northstar's support targets

Northstar gets better response targets than the general Enterprise policy:

| Severity | Northstar        |
|----------|------------------|
| P1       | 15 minutes, 24×7 |
| P2       | 1 hour           |
| P3       | 8 business hours |

05\_Northstar\_Logistics\_Enterpri...

Compare that with the general Enterprise policy:

```
General Enterprise:
P1 = 30 min
P2 = 2 hours
P3 = 1 business day
```

Therefore:

```
Northstar + P1
↓
15 minutes
```

**Not 30 minutes.**

This is a perfect test of source precedence.

## B. Northstar cancellation rule

This is probably the most important customer-specific override.

Northstar may cancel **any BOOKED shipment before pickup with no cancellation fee**, regardless of how long ago it was booked. 05\_Northstar\_Logistics\_Enterpri...

Compare that with the general SOP:

```
BOOKED > 30 minutes
→ INR 250
```

So:

```
General SOP
BOOKED + >30 min
→ ₹250
```



Northstar agreement  
BOOKED before pickup  
→ ₹0

The agent must choose **Northstar's agreement**.

This is exactly why the assessment asks:

Can Northstar cancel ORD-1001 without a cancellation fee?

The answer cannot be determined from the general SOP alone.

---

## C. Service credits

Northstar has a monthly aggregate service-credit cap of:

**INR 5,000**

Otherwise, the current service-credit SOP applies. 05\_Northstar\_Logistics\_Enterpri...

---

## D. Account contact

Dedicated CSM:

**Priya Mehta** 05\_Northstar\_Logistics\_Enterpri...

### Simple purpose

This PDF answers:

"Are Northstar's rules different from ParcelPilot's normal rules?"

Yes.

And those differences override the defaults.

---

## 6. 06\_LumenWorks\_Service\_Agreement.pdf

### What is this document?

This is the active customer-specific agreement for **LumenWorks**.

- Account: ACCT-002
- Plan: Growth

- Term: 1 March 2026 – 28 February 2027
- Status: Active 06\_LumenWorks\_Service\_Agreement

Again, this document exists to test whether the AI can identify **customer-specific rules**.

## A. LumenWorks support targets

| Severity | LumenWorks       |
|----------|------------------|
| P1       | 2 business hours |
| P2       | 4 business hours |
| P3       | 2 business days  |

There is also:

**No weekend or after-hours support coverage.** 06\_LumenWorks\_Service\_Agreement

So the AI shouldn't assume that every Enterprise customer or every customer receives 24×7 support.

## B. Cancellation

LumenWorks has **no special cancellation-fee waiver**.

Therefore:

LumenWorks  
↓  
No cancellation override  
↓  
Use current Cancellation & Service Credit SOP

06\_LumenWorks\_Service\_Agreement

## C. Failed-pickup service credit

This is the major LumenWorks-specific override.

Default SOP:

> 2 hours late  
→ lower of ₹500 or 10% shipment fee

## LumenWorks:

> 4 hours late  
AND  
carrier at fault  
AND  
customer not at fault  
→ ₹300 fixed credit

The agreement explicitly says this replaces the default threshold and credit amount.

06\_LumenWorks\_Service\_Agreement

## Simple purpose

This PDF answers:

**"Does LumenWorks have special support or service-credit rules?"**

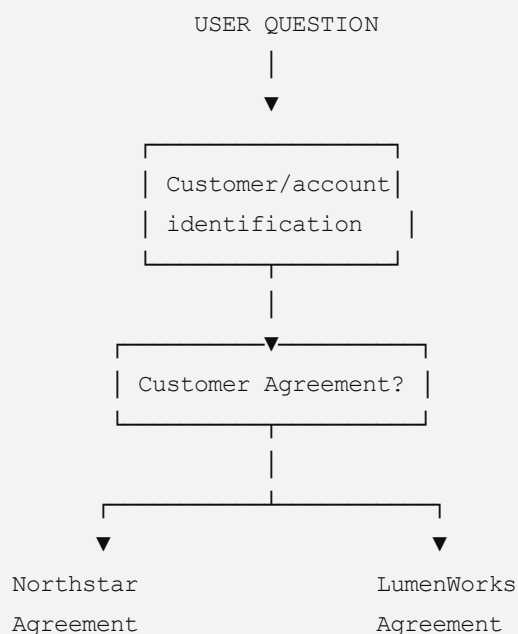
Yes — especially for failed pickups.

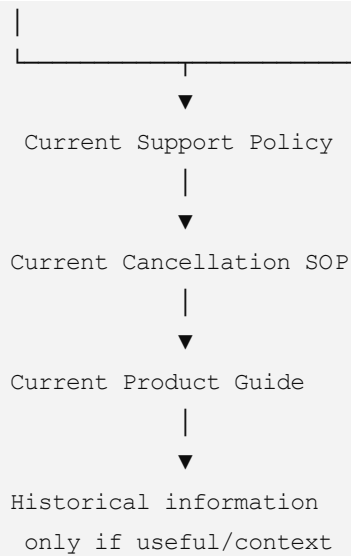
## Now put all six PDFs together

This is the **real intelligence problem** CalQuity is testing.

The documents aren't six independent PDFs.

They form a hierarchy:





And the most important rule is:

**Signed customer agreement → Current Support Policy → Current Product Documentation → Historical tickets/notes.**

01\_Support\_Policy\_v3\_CURRENT

## 🔥 The key lessons for our AI Agent

These six PDFs tell us exactly what our agent needs to be capable of.

### 1. Retrieval alone isn't enough

The agent can't simply ask:

"What does the retrieved document say?"

It needs to ask:

**"Which retrieved source is authoritative for this particular customer and situation?"**

### 2. Metadata is critical

Each document should carry metadata such as:

```
document
status
effective_date
customer
document_type
authority
```

For example:

```
Northstar Agreement
status = ACTIVE
customer = Northstar
authority = customer-specific
```

versus:

```
Support Policy v2
status = DEPRECATED
authority = historical
```

---

### 3. Customer context matters

The same question can produce different answers.

Example:

"Can I cancel this booked shipment after 30 minutes?"

For a normal customer:

Potential ₹250 fee

For Northstar:

₹0 cancellation fee

because Northstar's agreement overrides the general SOP.

03\_Cancellation\_and\_Service\_Cre...

05\_Northstar\_Logistics\_Enterpri...

---

### 4. Current status matters

For example:

```
BOOKED
↓
Cancellation may be possible

PICKED_UP
↓
Use return-to-origin
```

DELIVERED  
↓  
Cannot cancel

So the agent needs both **document knowledge** and **operational data**.

## 5. The AI must know when NOT to answer confidently

The SOP explicitly says not to promise a service credit when important facts are unknown, and conflicts should be identified before state-changing actions. 03\_Cancellation\_and\_Service\_Cre...

That means our agent should be capable of saying:

**"I don't have enough verified information to make that determination yet."**

That is not a weakness. For this assessment, it is a **trust feature**.



## What each PDF contributes to our eventual system

| PDF                      | Main role in the AI system                   |
|--------------------------|----------------------------------------------|
| v3 Current Policy        | Default severity + response rules            |
| v2 Deprecated Policy     | Tests freshness and source filtering         |
| Cancellation/SOP v4      | Cancellation + service-credit decision logic |
| Product Operations Guide | Product behavior + known issues              |
| Northstar Agreement      | Customer-specific overrides                  |
| LumenWorks Agreement     | Another customer's different overrides       |

So the data pack is essentially testing whether we can build an agent that understands:

**Context + Retrieval + Authority + Customer-specific rules + Structured data + Reasoning + Safety.**

And that fits very closely with the engineering framework you gave me earlier: **define the problem → specification → context engineering → tools → orchestration → guardrails → evaluation.**

The next logical step is **not coding yet**. We should now inspect

`ParcelPilot_Assessment_Data.xlsx`, because that is where the **accounts, orders, tickets, and snapshot time** live, and then map the relationships between the Excel data and these six PDFs.

Sources

Please provide a clear, straightforward summary of the document titled "1.01\_Support\_Policy\_v3\_CURRENT.pdf." Focus on explaining the main points of the policy or project in simple, easy-to-understand language. Highlight the key ideas, how the policy works, and its overall purpose without using technical jargon. Include a brief, practical example that illustrates how the policy might be applied in a real situation to help clarify its use. Keep the explanation concise, clear, and accessible for someone unfamiliar with the document or its field.

## Support Policy v3 — Simple Summary

**Document:** `01_Support_Policy_v3_CURRENT.pdf`

**Status:** Current

**Effective:** 1 May 2026

**Purpose:** Define how ParcelPilot classifies support problems and how quickly it should respond. `01_Support_Policy_v3_CURRENT`

### 1. Which rule should be followed?

The policy provides the **default rules**, but a customer's signed agreement can override them.

When information conflicts, follow this order:

1. **Signed customer agreement**
2. **Current Support Policy**
3. **Current product documentation**
4. **Historical tickets/internal notes** — these are only background because they may contain incorrect old guidance. `01_Support_Policy_v3_CURRENT`

## 2. How are problems classified?

ParcelPilot uses three severity levels:

- **P1 — Critical:** A serious problem such as a complete production outage or serious security incident, with no workaround.
- **P2 — High:** A major feature is unavailable or badly affected, but core operations can continue or a workaround exists.
- **P3 — Normal:** Smaller problems, how-to questions, configuration requests, or issues with limited impact.

01\_Support\_Policy\_v3\_CURRENT

## 3. How quickly should ParcelPilot respond?

| Customer plan | P1               | P2               | P3              |
|---------------|------------------|------------------|-----------------|
| Enterprise    | 30 min, 24×7     | 2 hours          | 1 business day  |
| Growth        | 2 business hours | 4 business hours | 2 business days |
| Standard      | 4 business hours | 1 business day   | 2 business days |

These are the **default** response targets. A customer's agreement may provide different targets.

01\_Support\_Policy\_v3\_CURRENT

## 4. What happens when something is urgent?

P1 problems should be **escalated immediately**.

If ParcelPilot has already missed its response target, the support system should clearly say that the target was breached and recommend escalation rather than hiding the problem.

01\_Support\_Policy\_v3\_CURRENT

### Practical example

Suppose an **Enterprise** customer has a complete production outage that prevents them from creating any shipments.

That is **P1 — Critical**.

Under the normal policy:

**ParcelPilot's first-response target is 30 minutes, 24×7.**

Because it is P1, the issue should also be **escalated immediately**.

However, if that customer has a signed agreement giving them a **15-minute P1 response target**, the customer's agreement takes priority over the standard 30-minute rule.



## Bottom line

The policy is basically a **default rulebook for support**:

**Identify how serious the problem is** → **check the customer's specific agreement**  
→ **determine the response deadline** → **escalate urgent or overdue problems**.

For our CalQuity AI agent, this document is especially important because the agent must **not blindly use the first rule it retrieves**; it needs to determine whether a customer-specific agreement overrides this policy.

No, I don't understand. This is the first time I am reading.  
Can you explain deeply and simply? I'd like you to explain briefly, in simple words, a clear explanation in detail

Absolutely. Let's forget the previous explanation and start **from zero**, as if you have never seen a support policy before.

The most important thing is to understand **why this document exists**, then the rules will make sense.

## Support Policy v3 — From Zero

01\_Support\_Policy\_v3\_CURRENT

### 1. First, what is ParcelPilot?

From the assessment context, **ParcelPilot is a logistics platform**.

Businesses use it to manage shipments.

For example:

```
A company
 ↓
creates shipment
 ↓
carrier picks up parcel
 ↓
parcel is delivered
```

Now imagine a customer has a problem:

"I cannot create shipments."

or

"A major feature isn't working."

or

"How do I upload a CSV?"

ParcelPilot's support team needs to decide:

**How serious is this problem?**

**How quickly should we respond?**

**Does this customer have special support rules?**

That is what this document is mainly about.

---

## 2. Think of the policy as a rulebook

Imagine you work in ParcelPilot's customer-support team.

A customer sends:

"Your system is completely down. We cannot create any shipments."

You cannot simply answer randomly.

The company needs a standard procedure.

The Support Policy says:

**First classify how serious the problem is. Then determine how quickly ParcelPilot needs to respond.**

So the document is basically a **support rulebook**.

It tells employees:

Customer has a problem

↓

How serious is it?

↓

P1 / P2 / P3

↓  
How quickly must we respond?  
↓  
Should we escalate?

### 3. What are P1, P2 and P3?

This is the most important part.

There are **three levels of problem severity**.

Think of them like:

P1 = Emergency  
P2 = Serious  
P3 = Normal

#### P1 — Critical

P1 means:

**Something extremely serious is happening and the customer may not be able to operate normally.**

The policy gives examples such as:

- complete production outage preventing all shipment creation
- confirmed security incident
- suspected credential exposure
- another situation causing immediate significant business risk with no workaround.

01\_Support\_Policy\_v3\_CURRENT

#### Example

Imagine Amazon-like shipping software is being used by a company.

Suddenly:

100% of shipment creation is broken.

The company cannot create **any shipments**.

That's potentially:

## P1 — Critical

Why?

Because the customer's business may effectively stop.

---

## 4. P2 — High

P2 means:

**Something important is broken, but the entire business is not completely stopped.**

The policy says this could be:

- a major feature being unavailable
- a feature being seriously degraded

but:

- core operations are still possible, **or**
- there is a workaround.

01\_Support\_Policy\_v3\_CURRENT

### Example

Imagine:

"Bulk upload isn't working."

But the customer can still create shipments **one at a time**.

That's serious because an important feature isn't working.

But the company isn't completely blocked.

So it could be:

## P2 — High

---

## 5. P3 — Normal

P3 is for smaller problems.

Examples from the policy include:

- minor defect

- how-to question
- configuration request
- issue with limited operational impact.

01\_Support\_Policy\_v3\_CURRENT

## Example

Customer asks:

"How do I configure my account?"

Nothing is broken.

They just need information.

That's:

**P3 — Normal**

---

## 6. Why do we need P1/P2/P3?

Because **not every customer problem deserves the same urgency.**

Imagine ParcelPilot receives these three requests:

### Customer A

"We cannot create any shipments."

### Customer B

"Bulk upload isn't working, but we can create shipments manually."

### Customer C

"How do I change my account settings?"

If ParcelPilot treated all three equally, that would be inefficient.

Instead:

```
Customer A → P1 → Emergency
Customer B → P2 → Serious
Customer C → P3 → Normal
```

Now support employees know which problem needs attention first.

---

## 7. But there is another important question

Once we know:

"This is P1."

we still need to ask:

"How quickly do we have to respond?"

This is where the next part of the document comes in.

## 8. Response time

The policy gives different response targets depending on the customer's **plan**.

There are three plans:

Enterprise  
Growth  
Standard

And there are three severity levels:

P1  
P2  
P3

So the company has a table telling support staff how quickly they should respond.

01\_Support\_Policy\_v3\_CURRENT

The default rules are:

| Customer plan | P1               | P2               | P3              |
|---------------|------------------|------------------|-----------------|
| Enterprise    | 30 minutes, 24×7 | 2 hours          | 1 business day  |
| Growth        | 2 business hours | 4 business hours | 2 business days |
| Standard      | 4 business hours | 1 business day   | 2 business days |

## 9. Let's understand this with one example

Suppose:

**Customer:** Enterprise

**Problem:** Complete shipment outage

We already determined:

Complete outage

↓

P1

Now look at the table:

Enterprise + P1

↓

30 minutes, 24×7

So ParcelPilot's default target is:

**Respond within 30 minutes.**

That's the basic idea.

---

## 10. But wait — there is an important exception

This is actually one of the most important ideas in the whole document.

The policy says:

A signed customer agreement may override the default support policy.

And when sources conflict, the order is:

1. Signed customer agreement
2. Current Support Policy
3. Current Product Documentation
4. Historical tickets/internal notes

01\_Support\_Policy\_v3\_CURRENT

This means the Support Policy is **not always the final answer**.

It is the **default answer**.

---

## 11. Why would a customer have different rules?

Imagine ParcelPilot has a very large customer called **Company X**.

Company X negotiated a special contract with ParcelPilot.

The contract says:

"For our company, P1 problems must receive a response within 15 minutes."

But the normal ParcelPilot policy says:

"Enterprise P1 = 30 minutes."

Which one should ParcelPilot follow?

**15 minutes.**

Why?

Because the customer's signed agreement has higher priority.

This is exactly why we need to check the customer agreement before blindly applying the general policy.

---

## 12. This becomes VERY important in our assessment

Remember the other PDF we looked at:

### Northstar Logistics Enterprise Agreement

It says Northstar gets:

```
P1 → 15 minutes
P2 → 1 hour
P3 → 8 business hours
```

05\_Northstar\_Logistics\_Enterpri...

But the general Support Policy says Enterprise customers get:

```
P1 → 30 minutes
P2 → 2 hours
P3 → 1 business day
```

01\_Support\_Policy\_v3\_CURRENT



So if the AI receives:

"Northstar has a P1 incident. How quickly must we respond?"

It should **not** answer:

"30 minutes."

It needs to recognize:

```
Northstar
 ↓
Customer-specific agreement exists
 ↓
Agreement overrides default policy
 ↓
P1 = 15 minutes
```

That is the kind of reasoning CalQuity is testing.

---

## 13. What about old information?

Now there is another interesting problem.

We have:

### Support Policy v3 — Current

and

### Support Policy v2 — Deprecated

The old policy contains different response times.

If an AI searches documents and finds both, what should it do?

It should use:

### v3

because v3 is the current policy.

The old v2 document is kept for historical reference and specifically says it must not be used for current requests. 02\_Support\_Policy\_v2\_DEPRECATED

So the AI needs to understand:

Found information

↓

Is it current?

↓

YES → usable

NO → historical/context only

This is why **retrieval alone isn't enough**.

---

## 14. What are historical tickets?

Imagine a support employee answered a customer two years ago:

"Your response target is 4 hours."

That answer might have been correct **at that time**.

But policies can change.

Therefore, an old ticket shouldn't automatically become today's rule.

The current policy explicitly says historical tickets and internal notes are **context only** and may contain incorrect past guidance. `01_Support_Policy_v3_CURRENT`

So:

Old ticket

↓

Useful for understanding history

↓

NOT automatically the current rule

---

## 15. What happens with P1?

The policy has another important rule.

If something is P1:

**Escalate immediately.**

And if the response target has already been missed, the support system should clearly say:

The response target has been breached.

and recommend escalation.

It should **not hide the fact that the target was missed**.

01\_Support\_Policy\_v3\_CURRENT

---

## 16. A complete real-world example

Let's put everything together.

### Situation

Northstar Logistics contacts ParcelPilot:

"We cannot create shipments. This is affecting our entire operation."

### Step 1 — Understand the problem

They cannot create shipments.

That is a potentially complete production outage.

### Step 2 — Classify severity

The policy says a complete production outage preventing shipment creation can be:

#### P1 — Critical.

01\_Support\_Policy\_v3\_CURRENT

### Step 3 — Find the customer's plan/agreement

Customer:

#### Northstar Logistics

There is a signed Northstar agreement.

### Step 4 — Check the customer's special rule

Northstar's agreement says:

#### P1 response = 15 minutes, 24×7.

05\_Northstar\_Logistics\_Enterpri...

### Step 5 — Compare with general policy

General Enterprise policy:

**30 minutes**

Northstar agreement:

**15 minutes**

## Step 6 — Choose the correct rule

Because the customer agreement has higher priority:

**15 minutes wins.**

## Step 7 — Escalate

Because it is P1:

**Escalate immediately.** 01\_Support\_Policy\_v3\_CURRENT

So the final reasoning is:

```
Northstar cannot create shipments
 ↓
Complete outage
 ↓
P1 Critical
 ↓
Northstar agreement exists
 ↓
Customer agreement overrides general policy
 ↓
Response target = 15 minutes, 24x7
 ↓
Immediate escalation
```

---

## 17. The simplest way to remember this PDF

Don't try to memorize every sentence.

Remember these **5 questions**:

### Question 1

**How serious is the problem?**

→ P1 / P2 / P3

### Question 2

**What is the customer's plan?**

→ Enterprise / Growth / Standard

### Question 3

**Does the customer have a special signed agreement?**

→ If yes, check it first.

#### **Question 4**

**Is the information current?**

→ Current policy beats old policy/history.

#### **Question 5**

**Is the issue urgent or already overdue?**

→ Escalate appropriately.

---

### **In one sentence**

**Support Policy v3 is ParcelPilot's default rulebook for deciding how serious a support problem is, how quickly ParcelPilot should respond, and when it should escalate—while recognizing that a customer's signed agreement can override the default rules.**

01\_Support\_Policy\_v3\_CURRENT

And for our AI project, the deeper lesson is:

**The AI should not just find an answer. It must determine which rule is applicable before giving the answer.**

That distinction is central to this assessment.

what are the best advice that you give to this company

Yes. If I were advising **CalQuity/ParcelPilot based on these documents**, I would give them one central recommendation:

**Don't build an AI that is merely good at answering questions. Build an AI support system that knows when it is allowed to answer, which source it should trust, what data it is allowed to see, and when a human must take over.**

That is the biggest weakness I would try to prevent.

## 1. Make "trust" the core product feature

The data pack deliberately contains conflicting and unreliable information:

- current vs deprecated policies
- customer-specific agreements
- current product documentation
- historical tickets that may be wrong
- operational data

The current policy itself establishes a source-precedence order, with signed customer agreements taking priority over the current support policy and historical tickets treated only as context.

01\_Support\_Policy\_v3\_CURRENT

So I would make the agent show **why it believes an answer**.

For example:

Answer: Northstar can cancel ORD-1001 without a fee.

Why:

- ✓ Order is BOOKED
- ✓ Northstar agreement applies
- ✓ Northstar agreement overrides the default cancellation rule

Sources:

1. Northstar Agreement
2. Cancellation SOP
3. Order data

Confidence: High

This is much better than simply saying:

"Yes, it can be cancelled."

Current enterprise AI research also increasingly emphasizes reliability, governance, and evaluating the complete agent workflow—not just the final answer.

HFS Rese... +1

## 2. Build a proper "source authority" system

I would not treat all documents equally.

Create something conceptually like:

Source

- |
- |— Current?
- |— Customer-specific?
- |— Effective date
- |— Document type
- |— Authority level

Then the agent can reason:

Northstar Agreement

↓

ACTIVE + customer-specific

↓

HIGH AUTHORITY

while:

Support Policy v2

↓

DEPRECATED

↓

DO NOT USE FOR CURRENT ANSWER

This is particularly important because v2 contains different response targets from v3.

02\_Support\_Policy\_v2\_DEPRECATED

**My advice:**

**Don't solve this only with prompting.**

Put source authority into the **retrieval/data layer**.

That makes the system much more reliable.

---

### 3. Separate "answer" from "action"

This is very important.

There are two completely different things:

**Informational action**

"What is Northstar's P1 response time?"

AI can answer.

## Operational action

"Escalate this ticket."

AI should **prepare** the action but not execute it immediately.

```
User
↓
Agent investigates
↓
Agent prepares escalation
↓
"Here is what I am going to do..."
↓
USER CONFIRMS
↓
Action executes
```

The assessment itself requires explicit confirmation before state-changing actions.

CalQuity AI Engineer — Job Desc...

I would keep this principle in the production system too.

Current research and enterprise guidance increasingly emphasize human oversight and strong authorization boundaries for agents that can take actions. **EC** Express Comp... +1

---

## 4. Put authorization in the tools, not the prompt

This is one of the strongest architectural recommendations I'd give them.

Bad:

```
System prompt:

"Don't show users another customer's information."
```

That's not sufficient.

Better:

```
User
↓
Agent
↓
Tool
↓
```



Authorization check

↓

Database

For example:

User = Northstar support user

Request:

"Show me LumenWorks orders."

↓

Authorization layer

↓

ACCESS DENIED

The assessment explicitly requires access control to be enforced at the data/tool layer rather than relying only on model instructions. CalQuity AI Engineer — Job Desc...

**I would consider this a production requirement, not just an assessment requirement.**

---

## 5. Don't make the AI "confident"; make it "appropriately certain"

This is a subtle but important difference.

Suppose the AI sees:

Pickup appears late.

That doesn't automatically mean:

Customer gets service credit.

The SOP says the system needs to know:

- whether the pickup was sufficiently late
- whether the carrier was at fault
- whether the customer caused the problem

If those facts are unknown, the agent should not promise the credit. 03\_Cancellation\_and\_Service\_Cre...

So I'd design responses like:

Known:

- ✓ Pickup was scheduled for 10:00
- ✓ Current time is 13:30

Unknown:

- ? Carrier fault
- ? Customer fault

Decision:

Cannot determine credit eligibility yet.

Next step:

Verify carrier status and fault information.

That's much safer than hallucinating a conclusion.

---

## 6. Create a dedicated conflict-resolution layer

I think this could become one of ParcelPilot's strongest features.

Imagine:

Current Policy:

₹250 cancellation fee

Northstar Agreement:

₹0 cancellation fee

The system shouldn't silently choose one.

It should understand:

CONFLICT DETECTED

↓

Who does the rule apply to?

↓

Northstar

↓

Customer-specific agreement

↓

Agreement wins

This is more sophisticated than ordinary RAG.

The agent isn't merely asking:

"What did I retrieve?"

It's asking:

**"Which retrieved information is legally/operationally applicable to this situation?"**

That is the kind of reasoning I would emphasize to CalQuity.

---

## 7. Build an evaluation system from the beginning

This is probably the biggest thing I'd add beyond the assessment.

Don't wait until the end and manually test five questions.

Create a test set such as:

50-100 realistic support questions

Categories:

Policy questions  
Customer agreement questions  
Order questions  
Ticket questions  
SLA questions  
Cancellation questions  
Service-credit questions  
Known-issue questions  
Conflicting-source questions  
Unauthorized-data questions  
Action/confirmation questions  
Unknown-information questions

Then measure:

### Answer correctness

Did it give the right answer?

### Source correctness

Did it use the correct document?

### Tool correctness

Did it choose the right tool?

## Permission correctness

Did it access only allowed data?

## Action safety

Did it ask for confirmation?

## Escalation correctness

Did it escalate when appropriate?

Modern agent evaluation is increasingly focused on the **entire execution path**—planning, retrieved context, tool use, permissions, intermediate decisions, and final outcome—not just whether the final sentence looks correct.  Snowfl...

---

## 8. Measure business value, not just "AI accuracy"

I would tell ParcelPilot:

Don't make your main KPI:

"Our LLM accuracy is 95%."

That's not enough.

I'd choose something like:

**Percentage of support requests resolved correctly without human rework.**

Then track:

```
Resolution rate
First-contact resolution
Incorrect-answer rate
Escalation accuracy
Average handling time
Human correction rate
Customer satisfaction
```

And especially:

**Wrong-action rate**

Because a wrong answer is bad.

A wrong action can be much worse.

## 9. Keep humans for judgment-heavy cases

I would **not** try to automate everything.

Use AI for:

```
Search
Summarization
Investigation
Classification
Calculations
Routine answers
Preparing actions
```

Use humans for:

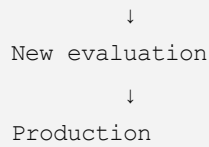
```
Exceptions
Conflicting evidence
High-risk actions
Unclear responsibility
Policy interpretation
Sensitive customer situations
```

That hybrid approach is also consistent with current enterprise CX research: AI-assisted human support remains a major deployment pattern, rather than fully autonomous agents being universally trusted.  Telus Di...

## 10. My longer-term recommendation: build a closed loop

If I were continuing the ParcelPilot product after this assessment, I'd build:

```
Customer interaction
 ↓
AI Agent
 ↓
Answer / Action
 ↓
Human correction
 ↓
Evaluation
 ↓
Failure analysis
 ↓
Improve retrieval / context / policies
```



In other words:

**AI → feedback → evaluation → improvement → AI**

Recent work on production support agents is moving toward exactly this kind of evaluation-driven, continuously improved workflow. [arXiv +1](#)

## If I were advising CalQuity in one page

My recommendations would be:

| Priority | Recommendation                                     |
|----------|----------------------------------------------------|
| 1        | Build source-aware reasoning                       |
| 2        | Enforce authorization outside the LLM              |
| 3        | Require confirmation for risky actions             |
| 4        | Detect conflicting/insufficient evidence           |
| 5        | Build systematic agent evaluation                  |
| 6        | Show evidence behind important answers             |
| 7        | Keep humans involved for judgment-heavy cases      |
| 8        | Monitor incorrect actions, not just hallucinations |
| 9        | Add proactive issue detection                      |
| 10       | Add continuous learning/improvement                |

## And one thing I would not recommend:

Don't tell them:

"Let's make it a multi-agent system."

just because multi-agent systems are fashionable.

First prove that **one well-designed agent with good tools, context, authorization, source handling, and evaluation** can solve the problem.

If one agent becomes genuinely overloaded with distinct responsibilities, *then* consider splitting responsibilities.

That's the engineering judgment I would want to demonstrate in your CalQuity submission.

**Your strongest pitch is therefore not "I built an AI chatbot."**

It's:

**"I designed a trustworthy support agent that can reason across policies, customer agreements, operational data, and product knowledge while respecting authorization, identifying uncertainty, and requiring human confirmation before taking action."**

That maps much more directly to the actual problems ParcelPilot gave you.

CalQuity AI Engineer — Job Desc...

Sources

 Sources

## What happens with P1?

The policy has another important rule.

If something is P1:

**Escalate immediately.**

And if the response target has already been missed, the support system should clearly say:

The response target has been breached.

Show more

Yes. Let's turn "**P1 escalation**" into an actual implementation roadmap for our ParcelPilot agent.

First, one important distinction:

**P1 escalation is not just a prompt instruction. It is a workflow.**

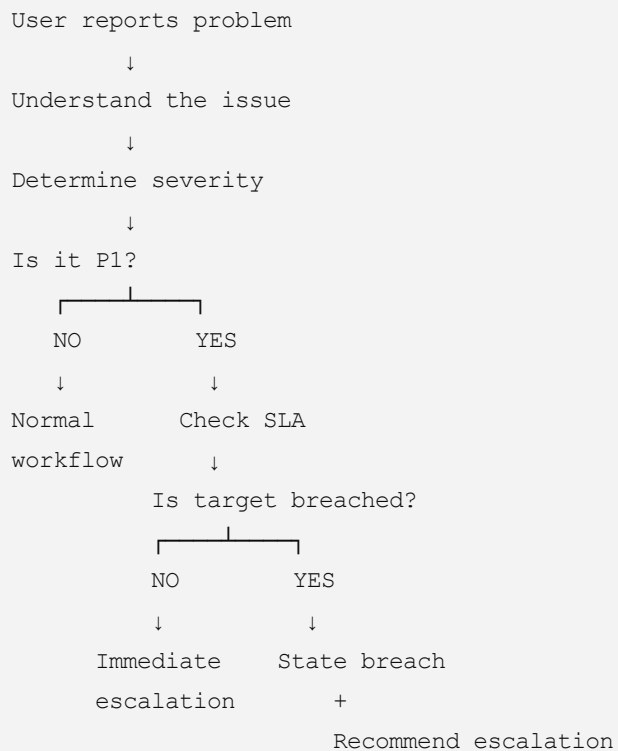
The current policy says P1 incidents should be escalated immediately, and if the response target has already been breached, the agent should clearly state the breach and recommend escalation.

01\_Support\_Policy\_v3\_CURRENT

## P1 Escalation Roadmap

### Goal

When a user reports a problem, our system should be able to:



## Phase 1 — Understand the problem

### Goal

Before deciding P1, the agent needs enough information to understand what happened.

Example:

"We cannot create any shipments."

The agent should identify:

Customer → Who?

Problem → Cannot create shipments



Impact → All shipments?  
Workaround → Is there one?  
Time → When did it start?

## Why?

Because **P1 isn't simply "the customer sounds angry."**

The policy defines P1 using the actual business impact, such as a complete production outage preventing shipment creation. 01\_Support\_Policy\_v3\_CURRENT

---

## Phase 2 — Classify severity

Create a dedicated classification step:

```
classify_severity()
```

Possible output:

JSON

```
{
 "severity": "P1",
 "reason": "Complete production outage preventing shipment creation",
 "evidence": ["order/account/system data"]
}
```

The classifier should distinguish:

### P1

Critical business impact.

### P2

Major problem, but operations can continue or a workaround exists.

### P3

Minor issue, question, configuration request, etc. 01\_Support\_Policy\_v3\_CURRENT

---

## Phase 3 — Identify the customer's SLA

This is where our **customer context** becomes important.

Don't immediately use the general Support Policy.

First ask:

```
Who is the customer?
 ↓
Do we have a signed agreement?
 ↓
YES → use customer agreement
NO → use current Support Policy
```

Why?

Because the policy explicitly says signed customer agreements can override the default policy. `01_Support_Policy_v3_CURRENT`

---

## Phase 4 — Determine the response target

For example:

**Normal Enterprise customer**

P1:

**30 minutes, 24×7** `01_Support_Policy_v3_CURRENT`

**Northstar**

Northstar's agreement overrides this:

**15 minutes, 24×7** `05_Northstar_Logistics_Enterpri...`

So our system should calculate:

```
customer = Northstar
severity = P1

↓
customer agreement found

↓
response target = 15 minutes
```

---

## Phase 5 — Calculate SLA status

Now we need to know:

## Has the response target already been missed?

Conceptually:

```
Incident created
 ↓
Current time
 ↓
Elapsed time
 ↓
Compare against response target
```

Output something like:

JSON

```
{
 "severity": "P1",
 "target": "15 minutes",
 "elapsed": "22 minutes",
 "sla_status": "BREACHED"
}
```

Or:

JSON

```
{
 "severity": "P1",
 "target": "15 minutes",
 "elapsed": "8 minutes",
 "sla_status": "WITHIN_TARGET"
}
```

## Important

The assessment says the workbook's **dataset snapshot time** should be used as the reference time for time-based questions.

CalQuity AI Engineer — Job Desc...

So don't casually use the computer's current time if the assessment question is based on the supplied dataset snapshot.

---

## Phase 6 — Decide what the agent should say

Now we have two different situations.

## Case A — P1 but NOT breached

Example:

```
P1
Target = 15 minutes
Elapsed = 5 minutes
```

Agent:

This is a P1 incident. Northstar's response target is 15 minutes, and the target has not yet been breached. The incident should be escalated immediately.

Why?

Because P1 itself requires immediate escalation.

01\_Support\_Policy\_v3\_CURRENT

---

## Case B — P1 AND breached

Example:

```
P1
Target = 15 minutes
Elapsed = 25 minutes
```

Now the response must explicitly say:

```
P1 incident
+
Response target = 15 minutes
+
25 minutes elapsed
+
Target breached
+
Recommend escalation
```

Not:

"Please wait."

Not:

"The issue is being handled."

Not:

"Support should respond soon."

The system should be transparent about the breach because the policy specifically says not to hide uncertainty or the breach.

01\_Support\_Policy\_v3\_CURRENT

## Phase 7 — Separate recommendation from action

This is where our earlier architecture matters.

The agent should **not automatically create an escalation simply because it detected P1** unless the action policy explicitly allows autonomous escalation.

For our assessment, I'd make the workflow:

```
P1 detected
 ↓
Agent prepares escalation
 ↓
Shows user:
- severity
- reason
- SLA target
- SLA status
- evidence
 ↓
Ask for confirmation
 ↓
User confirms
 ↓
create_escalation()
```

This follows the assessment's requirement that state-changing actions require explicit confirmation.

CalQuity AI Engineer — Job Desc...

## Phase 8 — Build the actual tool

We need a tool such as:

```
check_sla()
```

Input:

```
customer_id
ticket_id
severity
reference_time
```

### Output:

```
customer
severity
response_target
elapsed_time
sla_status
breach_duration
```

### Then:

```
create_escalation()
```

### Input:

```
ticket_id
priority
reason
```

But this tool should **only execute after confirmation**.

---

## Phase 9 — Test it

This is where I would spend serious time.

Create these tests:

### Test 1

```
P1 + within target
```

### Expected:

```
Immediate escalation recommended
No breach reported
```

### Test 2

P1 + target breached

**Expected:**

Clearly state SLA breach  
Recommend escalation

### Test 3

P2 + breached

Don't automatically classify it as P1.

The system must respect the severity definitions.

### Test 4

Northstar + P1

**Expected:**

15-minute target

Not the generic Enterprise 30-minute target.

### Test 5

Deprecated policy retrieved

**Expected:**

Do not use v2 as current policy

### Test 6

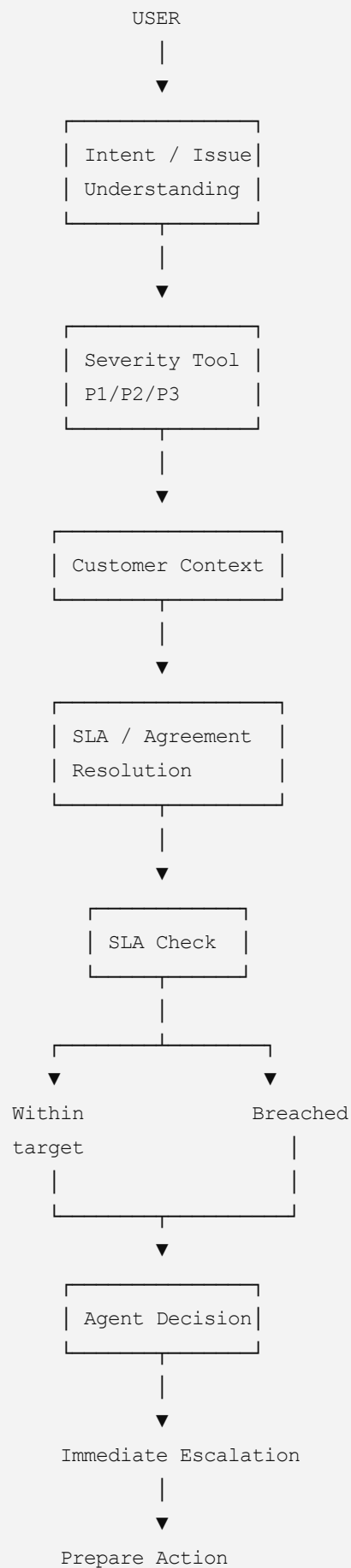
Insufficient information

**Expected:**

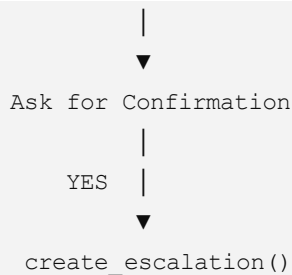
Ask for clarification  
or  
state that the available evidence isn't sufficient

# Final Architecture

I'd implement this as:







## The key design principle

Don't make the agent's reasoning:

**"P1 → escalate."**

Make it:

**"Understand the incident → establish P1 from evidence → identify the applicable customer SLA → determine whether the target has been breached → communicate the status honestly → recommend/prepare escalation → obtain confirmation before changing state."**

That is a much stronger implementation and demonstrates the exact engineering thinking CalQuity is looking for.

## Your implementation milestone

For this part of the project, your **definition of done** should be:

- ☐ P1/P2/P3 classification works
- ☐ Customer agreement takes precedence
- ☐ Current policy is distinguished from deprecated policy
- ☐ SLA target is calculated
- ☐ Breach is detected
- ☐ P1 is recommended for immediate escalation
- ☐ Breach is explicitly disclosed
- ☐ Evidence is shown
- ☐ User confirmation is required
- ☐ Escalation action is recorded
- ☐ At least 6 test cases pass

Once these work, **P1 escalation is no longer just a prompt—it is an actual engineered workflow.**

Sources

Sources

02\_Support\_Policy\_v2\_DEPRECATED.pdf can you explain about it indetailly

Yes. Let's study 02\_Support\_Policy\_v2\_DEPRECATED.pdf properly, from the beginning, as if this is your first time seeing it.

The important thing is that this document is **not the current rulebook**. It is an **older version kept for historical reference**. Understanding *why it exists* is actually important for our AI-agent project.

---

## 02\_Support\_Policy\_v2\_DEPRECATED.pdf

### 1. What is this document?

The document is called:

**ParcelPilot Support Policy v2**

It was effective from:

**1 January 2025**

But it was later replaced by:

**Support Policy v3, effective 1 May 2026.**

Its status is explicitly:

**DEPRECATED — DO NOT USE FOR CURRENT REQUESTS**

02\_Support\_Policy\_v2\_DEPRECATED

### What does "deprecated" mean?

Simply:

**This used to be the rule, but it is no longer the current rule.**

Think about a college syllabus.

Suppose:

2025 syllabus → old

2026 syllabus → current

You might keep the 2025 syllabus because you want to know what students were taught in 2025.

But if someone asks:

"What is the syllabus for my 2026 exam?"

you should **not use the 2025 syllabus**.

That's exactly what is happening here.

---

## 2. Why did ParcelPilot keep the old policy?

The document says it is intentionally retained for **historical reference**.

02\_Support\_Policy\_v2\_DEPRECATED

That means the old policy can still be useful for questions such as:

"What were the support targets last year?"

or

"Why did an old ticket say the response time was 4 hours?"

But it should **not** be used to answer:

"What is the current response target?"

This distinction is extremely important for our AI system.

---

## 3. What does v2 actually contain?

The document mainly defines:

1. **P1**
2. **P2**
3. **P3**
4. Response targets for different customer plans.

02\_Support\_Policy\_v2\_DEPRECATED

So conceptually:

```
Customer has a problem
 ↓
How serious?
 ↓
P1 / P2 / P3
 ↓
How quickly should ParcelPilot respond?
```

That is the purpose of this policy.

---

## 4. P1 — Critical

The old policy says P1 covers:

Complete production outages and severe security incidents.

02\_Support\_Policy\_v2\_DEPRECATED

In simple language:

**P1 = something extremely serious.**

For example:

```
ParcelPilot completely stops working
 ↓
Customer cannot operate
 ↓
P1
```

Or:

```
Serious security incident
 ↓
P1
```

---

## 5. P2 — High

The old policy describes P2 as:

Major feature degradation.

02\_Support\_Policy\_v2\_DEPRECATED

Simple meaning:

Something important is badly affected, but it isn't necessarily a complete system failure.

For example:

Major feature broken

↓

But the entire system isn't completely down

↓

P2

## 6. P3 — Normal

The old policy says P3 covers:

Minor issues and questions.

02\_Support\_Policy\_v2\_DEPRECATED

So:

Small issue

How-to question

General support question

↓

P3

## 7. The old response-time table

This is where the document becomes especially useful for understanding why it is dangerous to use old information.

The old policy had these targets:

| Customer Plan | P1               | P2              | P3              |
|---------------|------------------|-----------------|-----------------|
| Enterprise    | 1 hour           | 4 hours         | 2 business days |
| Growth        | 4 business hours | 1 business day  | 3 business days |
| Standard      | 8 business hours | 2 business days | 3 business days |

02\_Support\_Policy\_v2\_DEPRECATED

Let's understand this rather than just memorize the table.

## 8. Example: Enterprise customer

Suppose an Enterprise customer has a P1 problem.

According to **v2**:

```
Enterprise
+
P1
↓
1 hour
```

So the old response target was:

**1 hour**

But remember:

**This is no longer the current target.**

---

## 9. Example: Growth customer

Suppose a Growth customer has a P1 issue.

Under v2:

```
Growth
+
P1
↓
4 business hours
```

So the old policy expected a response within:

**4 business hours**

Again, this is historical information.

---

## 10. Example: Standard customer

Suppose a Standard customer has a P1 problem.

Under v2:

```
Standard
+
P1
↓
8 business hours
```

That's the old target.

---

## 11. Why is this document dangerous for an AI?

This is actually the **important part for our project**.

Imagine we put both documents into a vector database:

```
v2 → old policy
v3 → current policy
```

A user asks:

"What is the Enterprise P1 response time?"

The search system might retrieve **both** documents because both contain:

Enterprise + P1 + response time.

A basic RAG system might see:

```
v2:
Enterprise P1 = 1 hour

v3:
Enterprise P1 = 30 minutes
```

and produce something like:

"The response target is 1 hour."

That would be **wrong for a current request**.

The problem isn't that retrieval failed.

The problem is that the AI **failed to understand document authority and freshness**.

---

## 12. This is why "retrieval" is not enough

This document teaches us an important AI engineering lesson:

```
Search
 ↓
Find relevant documents
 ↓
NOT automatically:
"Trust everything found"
```

Instead:

```
Search
 ↓
Retrieve candidates
 ↓
Check document status
 ↓
Check effective date
 ↓
Determine authority
 ↓
Select applicable source
 ↓
Answer
```

For v2:

```
Retrieved
 ↓
Status = DEPRECATED
 ↓
Current request?
 ↓
YES
 ↓
DO NOT USE
```

That's the behavior we want.

---

## 13. Compare v2 and v3

This comparison makes the problem much clearer.

### Enterprise P1

**v2:**



1 hour

02\_Support\_Policy\_v2\_DEPRECATED

**v3:**

30 minutes, 24×7

01\_Support\_Policy\_v3\_CURRENT

So the rule changed.

---

## Enterprise P2

**v2:**

4 hours

**v3:**

2 hours

So again, the current policy is different.

---

## Enterprise P3

**v2:**

2 business days

**v3:**

1 business day

Again, changed.

This proves why the system cannot simply find a relevant paragraph and use it.

---

## 14. What should our AI do?

Suppose the user asks:

## "What is the current Enterprise P1 response target?"

The AI might retrieve:

Document A:  
Support Policy v2  
Status: DEPRECATED  
P1 = 1 hour

Document B:  
Support Policy v3  
Status: CURRENT  
P1 = 30 minutes

The AI should reason:

v2  
↓  
Deprecated  
↓  
Historical only

v3  
↓  
Current  
↓  
Applicable

Therefore:  
30 minutes, 24×7

That is the correct behavior.

---

## 15. But what if the user asks a historical question?

Now consider:

### "What was the Enterprise P1 response target under the 2025 policy?"

Now v2 becomes relevant.

Because:

Question asks about 2025  
↓  
v2 was effective in 2025  
↓

v2 is relevant  
↓  
Answer = 1 hour

So **deprecated** doesn't mean useless.

It means:

**Don't use it for current decisions.**

That's a much better mental model.

---

## 16. What about an old support ticket?

Imagine an old ticket from 2025 says:

"Enterprise P1 should receive a response within 1 hour."

That ticket might be perfectly consistent with v2.

But today, if someone asks:

"What is the current Enterprise P1 target?"

we shouldn't use that old ticket.

Why?

Because the current policy changed.

The current v3 policy itself tells us historical tickets and internal notes may contain incorrect past guidance and should only be treated as context.

01\_Support\_Policy\_v3\_CURRENT

---

## 17. What should we store as metadata?

When we load this PDF into our AI system, I would **not** store just the text.

We should associate metadata such as:

Document:  
Support Policy v2

Status:  
DEPRECATED

```
Effective:
1 January 2025

Superseded by:
Support Policy v3

Use for:
Historical reference

Current decision:
NO
```

Then the agent can reason about the document.

This is **context engineering**.

We're giving the model not just:

"Here is some text."

We're giving it:

"Here is text + what it means + when it applies + when it should not be trusted."

---

## 18. What is the biggest lesson from this PDF?

If you're building the ParcelPilot AI agent, **don't treat document retrieval as document truth**.

A retrieved document can be:

```
Relevant
but outdated.
```

```
Relevant
but customer-specific.
```

```
Relevant
but historical.
```

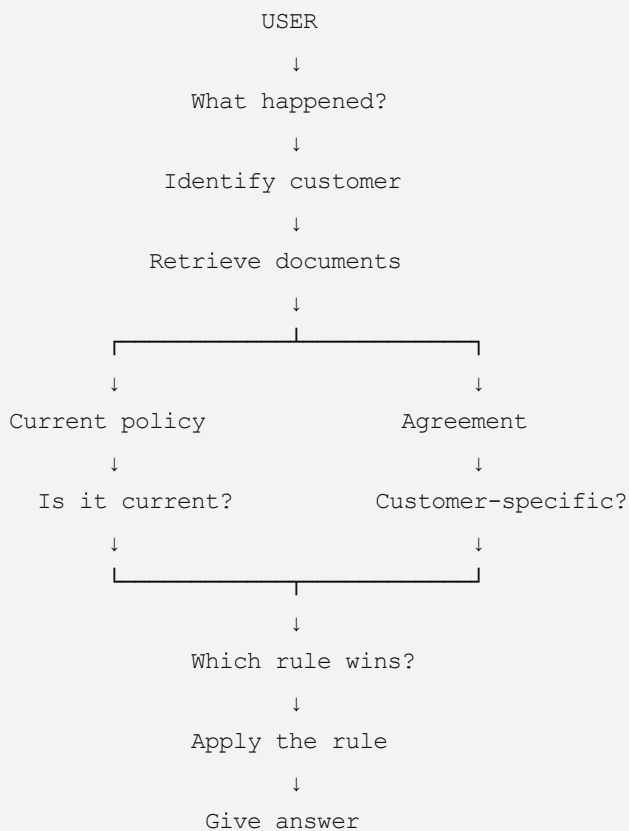
```
Relevant
but lower authority.
```

```
Relevant
but contradicted by a stronger source.
```

The agent needs to determine **applicability**, not just relevance.

## 19. How this fits our overall project

Think of the six PDFs like this:



And v2 exists specifically to test the part:

**"Is this information current?"**

## 20. The one thing I want you to remember

Don't remember the entire v2 table.

Remember this:

**Support Policy v2 is the OLD ParcelPilot support rulebook. It tells us what the rules used to be, but it must not be used to answer current support questions.**

And for our AI agent:

**The agent must recognize that v2 is deprecated and prevent it from overriding current information.**

That single concept is more important for the project than memorizing the individual response times.

Relevant but contradicted by a stronger source. explain

Yes. This phrase is **very important for our ParcelPilot project**, so let's understand it slowly.

## "Relevant but contradicted by a stronger source"

It means:

The information we found is related to the user's question, but another source has higher authority, so we should not use the weaker information as the final answer.

This is a common problem in RAG systems: retrieval finds information based on relevance, but **relevance does not automatically tell us which source should win when sources disagree.**  ScienceDir... +1

### 1. First understand "relevant"

**Relevant** simply means:

"This document talks about the thing the user asked about."

Imagine I ask:

**"What is the Enterprise P1 response time?"**

Our system searches its documents.

It finds:

#### Document A — v2

Enterprise P1 = 1 hour

This is **relevant** because it talks about:

- Enterprise
- P1
- response time

So the search system is correct to find it.

But that doesn't mean we should use it.

---

## 2. Now "contradicted"

Suppose the system also finds:

### Document B — v3

Enterprise P1 = 30 minutes

Now we have:

v2 → 1 hour

v3 → 30 minutes

These two pieces of information **contradict each other**.

They cannot both be the current Enterprise P1 response target.

---

## 3. Now "stronger source"

We ask:

**Which document should we trust more?**

Look at their status.

### v2

Status: DEPRECATED

It explicitly says:

Do not use for current requests. 02\_Support\_Policy\_v2\_DEPRECATED

### v3

Status: CURRENT  
Effective: 1 May 2026

01\_Support\_Policy\_v3\_CURRENT

Therefore:

v2  
↓  
Relevant  
BUT  
Deprecated  
↓  
WEAKER for a current question

v3  
↓  
Relevant  
AND  
Current  
↓  
STRONGER

So the agent should answer:

**30 minutes, 24×7.**

Not 1 hour.

## 4. Why is v3 "stronger"?

Because in this particular system, **the documents have different authority.**

The current Support Policy says the source order is:

1. Signed customer agreement  
↓  
2. Current Support Policy  
↓  
3. Current Product Documentation  
↓  
4. Historical tickets / notes

01\_Support\_Policy\_v3\_CURRENT

So "stronger" doesn't mean:



"This document sounds better."

It means:

**According to the business rules, this source has greater authority for this particular question.**

That's a very important distinction.

---

## 5. An even better example: Northstar

This is where our assessment becomes really interesting.

Suppose the user asks:

**"What is Northstar's P1 response time?"**

Our system might retrieve **three sources**.

### Source 1 — v2

Enterprise P1 = 1 hour

But:

DEPRECATED

✗ Don't use for current answer.

### Source 2 — v3

Enterprise P1 = 30 minutes

Current.

✓ Good default rule.

### Source 3 — Northstar Agreement

Northstar P1 = 15 minutes

And this is a **signed customer agreement**.

The current policy says customer agreements override the default policy.

01\_Support\_Policy\_v3\_CURRENT

Therefore:

```
v2
1 hour
 ↓
DEPRECATED
 ↓
Reject

v3
30 minutes
 ↓
CURRENT
 ↓
Valid default

Northstar Agreement
15 minutes
 ↓
CUSTOMER-SPECIFIC
 ↓
STRONGEST
```

**Final answer:**

**15 minutes, 24×7.**

05\_Northstar\_Logistics\_Enterpri...

---

## 6. So "stronger source" depends on context

This is extremely important.

There is **no universal rule saying the newest document always wins**.

For ParcelPilot, the source hierarchy is explicitly defined.

For example:

**Question:**

"What is the general Enterprise P1 target?"

Current Support Policy wins:

**30 minutes.** 01\_Support\_Policy\_v3\_CURRENT

**Question:**

"What is Northstar's P1 target?"

Northstar's signed agreement wins:

**15 minutes.** 05\_Northstar\_Logistics\_Enterpri...

**Question:**

"What was the Enterprise P1 target under the 2025 policy?"

Now v2 becomes relevant because we're asking about the **historical period** covered by that document.

**1 hour.** 02\_Support\_Policy\_v2\_DEPRECATED

So the agent has to understand:

**Question + customer + time + source authority**

not just:

**Question → search → answer.**

---

## 7. This is the difference between basic RAG and better RAG

### Basic RAG

User question

↓

Search

↓

Top documents

↓

LLM

↓

Answer

Problem:

The retrieved documents may disagree.

Research on RAG specifically identifies this as a reliability problem: systems can retrieve multiple relevant but conflicting sources and still struggle to resolve which evidence should control the answer.  Google Resea... +1

---

## Our ParcelPilot approach

We want:

```
graph TD
 A[User question] --> B[Retrieve sources]
 B --> C[Check metadata]
 C --> D[Check customer]
 D --> E[Check date/status]
 E --> F[Detect conflicts]
 F --> G[Apply source hierarchy]
 G --> H[Select applicable evidence]
 H --> I[Reason]
 I --> J[Answer]
```

That's much better.

---

## 8. Think of it like a courtroom

This analogy may make it easier.

Imagine two people give you different statements.

### Person A

"I heard the rule is 1 hour."

### Person B

"Here is the current signed agreement saying 15 minutes."

Both statements are **relevant**.

But they don't have equal authority.

You wouldn't say:

"Both said something, so I'll average them and say 37.5 minutes."



You ask:

**"Which evidence has authority?"**

That's what our AI needs to do.

---

## 9. Never average conflicting rules

This is a critical AI-agent principle.

Suppose:

Source A → ₹250 fee

Source B → ₹0 fee

The AI must **not** say:

"The fee is approximately ₹125."

That's nonsense.

It needs to determine:

Why do they differ?

↓

Different customer?

Different date?

Different version?

Different authority?

↓

Which rule applies?

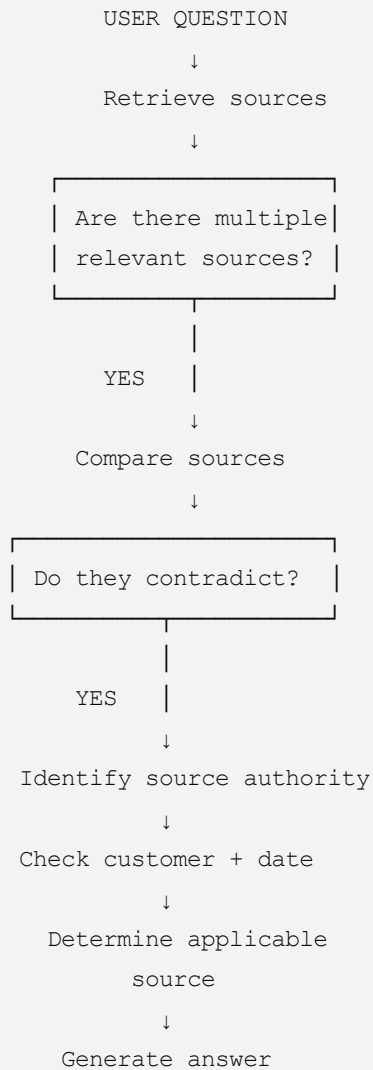
If the system cannot determine which source wins, it should **surface the conflict rather than invent a resolution**. Conflict-aware RAG research similarly emphasizes explicit

conflict detection and source credibility rather than blindly combining retrieved evidence.

Google Resea... +1

## 10. What should our AI actually do?

Let's turn this into an engineering workflow.



## 11. What if there is no clear stronger source?

This is another important situation.

Suppose:

Document A → ₹500

Document B → ₹300

And we don't know:

- which is current
- which customer it applies to
- which document has higher authority.

Then the AI shouldn't confidently choose ₹300 or ₹500.

It should say something like:

"I found conflicting information in the available sources, and I cannot determine which rule applies with sufficient confidence. This needs verification before an action is taken."

This is especially important for ParcelPilot because the SOP explicitly says that when data conflicts, the conflict should be identified and verification requested before a state-changing action. 03\_Cancellation\_and\_Service\_Cre...

---

## 12. The key distinction

Remember these three words:

### Relevant

Does this information relate to the question?

### Reliable

Is this source trustworthy/current/authoritative?

### Applicable

Does this rule actually apply to this customer and situation?

These are **not the same thing**.

A document can be:

Relevant ✓

Reliable-ish ✓

Applicable ✗

For example:

**Support Policy v3** is relevant and reliable, but for Northstar's special P1 response time, the **Northstar agreement is the applicable higher-priority rule**.

## ★ The concept I want you to remember

When you build our agent, don't think:

"Find the most relevant document."

Think:

"Find the relevant evidence, then determine which evidence has authority and actually applies to this situation."

That's the difference between a simple document chatbot and a **reliable AI support system**.

And this is exactly why your earlier principle — **"Context Engineering matters more than simply writing bigger prompts"** — matters here. The agent needs context about **document status, effective date, customer, authority, and applicability**, not just the raw text.

Sources

 Sources

why we need old data

Yes — and this is an important question. **We do not need old data because it is "better."** **We need it because it can provide historical context and, more importantly for this assessment, it tests whether our AI knows when NOT to use information.**

Let's separate the ideas.

### 1. Old data is not the same as current truth

Take our `Support Policy v2`.

It says:

Enterprise P1 = **1 hour**

But v2 was superseded by v3. `02_Support_Policy_v2_DEPRECATED`

The current v3 says:



Enterprise P1 = **30 minutes, 24×7.**

01\_Support\_Policy\_v3\_CURRENT

So if someone asks:

"What is the current Enterprise P1 response time?"

We use **v3**, not v2.

Therefore:

**Old data ≠ current answer.**

---

## 2. Then why keep the old data?

Because sometimes the question itself is about the past.

For example:

"What was the Enterprise P1 response target in 2025?"

Now v2 becomes useful.

Why?

Because v2 was effective from **1 January 2025** and contains the rules from that period.

02\_Support\_Policy\_v2\_DEPRECATED

So:

Current question

↓

Current source

Historical question

↓

Historical source

The **question's time period determines whether old information is relevant.**

---

## 3. Old data also explains past decisions

Imagine a support employee handled a ticket in 2025.

They wrote:

"Enterprise P1 response target is 1 hour."

Today someone sees that ticket and asks:

"Why did the employee say 1 hour?"

Without the old policy, that answer might be confusing.

With v2:

```
2025
↓
Support Policy v2
↓
Enterprise P1 = 1 hour
```

Now we understand why the employee gave that answer.

So old information can help explain **historical decisions**.

---

## 4. It can also help detect mistakes

This is actually very useful for an AI system.

Suppose:

```
Old policy → 1 hour
Current policy → 30 minutes
```

A support ticket from last year says:

"Your response target is 1 hour."

The AI can understand:

"That statement may have been correct under the old policy, but it isn't the current rule."

This prevents the AI from incorrectly treating old information as today's policy.

---

## 5. But there is an even bigger reason in THIS assessment

The CalQuity assessment intentionally tells us that the source base is imperfect.

It includes:

- outdated documents
- customer-specific agreements
- potentially incorrect historical ticket resolutions.

CalQuity AI Engineer — Job Desc...

Why would they do that?

Because they want to see whether you can build an AI system that handles **messy real-world information**.

Real companies don't have:

```
Perfect documents
+
One database
+
One correct answer
```

They have:

```
Old policies
New policies
Customer contracts
Tickets
Notes
Databases
Conflicting information
```

The engineer's job is to build a system that can **reason about those differences**.

---

## 6. Think about Google Maps

Imagine you have:

```
Old address
New address
```

You don't delete the old address from history just because the business moved.

The old address may still answer:

"Where was the business located in 2024?"

But if someone asks:

"Where is it now?"

you need the new address.

Same idea:

```
Old policy
 ↓
Useful for historical context

Current policy
 ↓
Useful for current decisions
```

---

## 7. What should our AI do with old data?

This is the important engineering part.

We shouldn't simply delete old documents.

Instead, give them metadata:

```
document = Support Policy v2
status = DEPRECATED
effective_date = 2025-01-01
superseded_by = Support Policy v3
```

Then our retrieval/decision system can understand:

```
Retrieved v2
 ↓
Is user asking about 2025?
 |
YES → v2 may be relevant
 |
NO
 ↓
Current request
 ↓
Do not use v2 as governing rule
```

---

## 8. The critical distinction

This is what I want you to remember:

### We don't ask:

"Is this data old?"

We ask:

"Is this data applicable to the user's question?"

Old data can be:

**Relevant** ✓

but

**Not applicable to the current decision** ✗

For example:

"What was the policy in 2025?"

v2 → applicable ✓

"What is the policy today?"

v2 → not applicable ✗

---

## 9. And don't confuse "old data" with "bad data"

This is another important distinction.

Old data isn't necessarily wrong.

It may have been **correct at the time**.

For example:

2025:

v2 → 1 hour

could have been correct.

Then:

2026:  
v3 → 30 minutes

became the new rule.

So the correct interpretation is:

**"v2 is historically valid but currently superseded."**

Not:

**"v2 is fake."**

The document itself says it is retained for historical reference and must not be used for current requests. 02\_Support\_Policy\_v2\_DEPRECATED

## 10. Why CalQuity cares about this

This is actually testing whether you understand **AI engineering beyond simple RAG**.

A basic system thinks:

```
Question
↓
Search
↓
Relevant document
↓
Answer
```

A better system thinks:

```
Question
↓
Search
↓
Relevant sources
↓
When was each source valid?
↓
Who does it apply to?
↓
Which source has higher authority?
↓
Do sources conflict?
↓
```

Which rule applies?

↓

Answer

That's the real challenge.

## One sentence to remember

**We keep old data because it can explain the past and answer historical questions, but our AI must know that old information should not automatically control current decisions.**

And in the CalQuity assessment, **the fact that old and potentially unreliable information exists is deliberate**: they're testing whether your agent can distinguish **relevance from applicability and authority**.

CalQuity AI Engineer — Job Desc...

Document:  
Support Policy v2

Status:  
DEPRECATED

Effective:  
1 January 2025

Superseded by:  
Support Policy v3

Show more

Yes — but there is an important distinction here.

**The metadata I showed is NOT necessarily data that already exists in the Excel sheet.** I was describing metadata that **we should attach to the PDF when we build the AI system.**

Let's separate the two.

# 1. The PDF itself contains information

For `Support_Policy_v2` we know from the PDF:

```
Document:
Support Policy v2

Status:
DEPRECATED

Effective:
1 January 2025

Superseded by:
Support Policy v3

Purpose:
Historical reference
```

Those facts come from the PDF itself. `02_Support_Policy_v2_DEPRECATED`

---

## 2. What is metadata?

Metadata means:

**Information that describes another piece of information.**

Think about a photo.

The photo is:

```
family.jpg
```

Metadata might be:

```
File name: family.jpg
Date: 2026-08-22
Location: Hyderabad
File type: JPEG
```

The metadata is **not the actual photograph**.

It describes the photograph.

Same thing with our PDF.

The actual PDF contains:



Enterprise P1 = 1 hour

Metadata describes the PDF:

```
status = deprecated
effective_date = 2025-01-01
document_type = support_policy
```

### 3. What about the Excel sheet?

This is where your question is very good.

The **Excel file is primarily the operational data source**, not simply "metadata."

The assessment says the workbook contains:

- account data
- order data
- ticket data

and also has a **README sheet containing the dataset snapshot time**.

CalQuity AI Engineer — Job Desc...

So conceptually:

```
PDFs
|
├─ Actual business knowledge
| ├── Policies
| ├── SOP
| ├── Product information
| └─ Customer agreements
|
└─ We attach metadata
 ├── status
 ├── effective date
 ├── customer
 └─ authority
```

Whereas:

```
Excel
|
├─ Accounts
├─ Orders
└─ Tickets
```

└─ README  
└─ Dataset snapshot time

The Excel records are **actual business/operational data** that the agent needs to query.

## 4. But Excel can also contain metadata

This is the subtle part.

A file can contain **both data and metadata**.

For example, suppose the Excel has:

| Order ID | Customer  | Status | Shipment Fee |
|----------|-----------|--------|--------------|
| ORD-1001 | Northstar | BOOKED | ₹800         |

These are **business data**.

But suppose the README says:

Dataset snapshot: 21 August 2026

That is metadata about the dataset.

It tells us:

"This data represents the state of ParcelPilot at this particular point in time."

So:

```
Excel
├─
├─ Business data
│ ├── accounts
│ ├── orders
│ └─ tickets
├─
└─ Dataset metadata
 └─ snapshot time
```

## 5. There is another layer: metadata WE create

This is what I was talking about earlier.

When we load the PDFs into our retrieval system, we can create metadata ourselves.

For example:

Python

Run

```
{
 "document": "Support Policy v2",
 "status": "deprecated",
 "effective_date": "2025-01-01",
 "superseded_by": "Support Policy v3",
 "document_type": "support_policy"
}
```

This isn't something the Excel sheet needs to provide.

**We derive it from the PDF and store it alongside the PDF chunks.**

---

## 6. Why do we need to create this metadata?

Imagine we split the PDF into chunks.

One chunk might simply contain:

Enterprise P1 = 1 hour.

If we put only that text into a vector database, the AI may retrieve it later and think:

"Great! Enterprise P1 is 1 hour."

It might not know that the document is deprecated.

So instead we store:

Chunk:

"Enterprise P1 = 1 hour"

Metadata:

status = DEPRECATED

effective\_date = 2025-01-01

document = Support Policy v2

Now the retrieval system has both:

**content + context**

## 7. This is exactly where Context Engineering comes in

You told me earlier:

**Context Engineering matters more than simply writing bigger prompts.**

This is a perfect example.

Without metadata:

LLM receives:

"Enterprise P1 = 1 hour"

With metadata:

LLM receives:

Document: Support Policy v2

Status: DEPRECATED

Effective: 1 Jan 2025

Superseded by: v3

Content:

Enterprise P1 = 1 hour

Now the model has the context necessary to understand:

"This information is historical, not the current rule."

---

## 8. There are actually THREE kinds of information in our system

This distinction will help you a lot.

### A. Document content

**What does the document say?**

Example:

Enterprise P1 = 1 hour.

### B. Document metadata

## What describes the document?

Example:

```
Status = DEPRECATED
Effective = 2025
Customer = none
Document type = policy
```

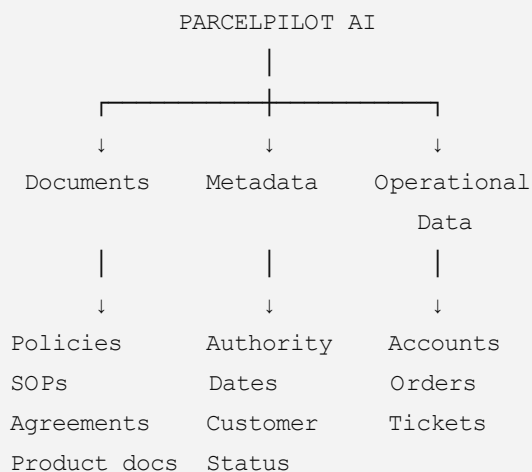
## C. Operational data

### What is actually happening in the business?

Example:

```
Order ORD-1001
Customer = Northstar
Status = BOOKED
```

So:



## 9. And this is why the AI needs both PDFs AND Excel

Suppose the user asks:

**"Can Northstar cancel ORD-1001 without a fee?"**

The PDF alone isn't enough.

We need:

**Excel**

Find:

```
ORD-1001
↓
Customer = Northstar
↓
Status = BOOKED
```

## Northstar Agreement

Find:

Northstar can cancel any BOOKED shipment before pickup without a cancellation fee.

05\_Northstar\_Logistics\_Enterpri...

## Cancellation SOP

Find the general rule:

BOOKED shipments after 30 minutes normally have a ₹250 fee, unless the customer agreement waives it.

03\_Cancellation\_and\_Service\_Cre...

Then the agent reasons:

```
ORD-1001
↓
Northstar
↓
BOOKED
↓
Northstar agreement applies
↓
Agreement overrides general SOP
↓
₹0 cancellation fee
```

That's the real AI workflow.

---

## 10. One correction to your understanding

Don't think:

"Excel = metadata."

That's too broad.

Think:

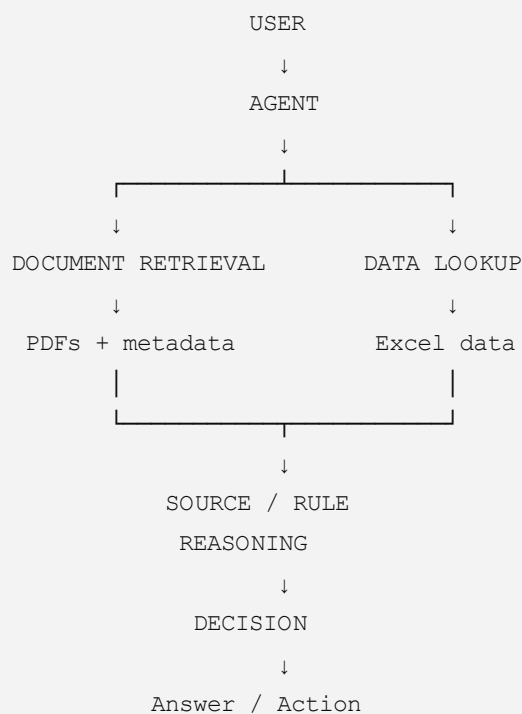
"Excel contains operational/business data, plus some metadata about the dataset."

And:

"The PDFs contain business rules/knowledge, and we should attach metadata describing their status, authority, dates, and applicability."

That distinction is important when we design the architecture.

**Our eventual architecture should therefore look like:**



**So your understanding is close, but don't call the Excel workbook "the metadata."** It is primarily the **structured operational data source**; the README's snapshot time is metadata, and the PDF metadata is something we'll explicitly model to help the agent decide which information it should trust and apply.

Sources

in the pdf the information is not so big how can we create it frist

Exactly. And this is a good question because **our PDFs are tiny**. We should not over-engineer the document system just because we are building a RAG/agent project.

The right approach is:

**First understand the amount and structure of the data. Then choose the simplest retrieval architecture that is sufficient.**

For these PDFs, I would **not start by building a complicated vector database**.

---

## 1. First: how much information do we actually have?

Our six PDFs are very small.

For example, Support Policy v3 is only a handful of rules:

```
Source precedence
P1 / P2 / P3 definitions
Response targets
Escalation rule
```

The other PDFs are similarly small.

So the total document knowledge is probably small enough that we can load the **entire document set into memory** during a prototype.

That changes our design decision.

---

## 2. Don't start with embeddings

A common beginner approach is:

```
PDF
↓
Split into 100 chunks
↓
Generate embeddings
↓
Vector database
↓
Similarity search
↓
LLM
```



For a huge knowledge base, this makes sense.

But for our assessment:

```
6 small PDFs
+
small amount of text
```

Building a sophisticated vector database **first** adds complexity without solving a real problem.

---

### 3. What should we do first?

I'd build this in **three stages**.

#### Stage 1 — Simple document loader

First, read the PDFs.

```
PDF files
 ↓
Extract text
 ↓
Document objects
```

Each document becomes something like:

Python

Run

```
{
 "id": "support_policy_v2",
 "text": "...",
 "metadata": {
 "status": "deprecated",
 "effective_date": "2025-01-01",
 "document_type": "support_policy"
 }
}
```

At this stage:

**No embeddings.**

**No agent.**

**No complicated RAG.**

Just make sure we can reliably read the documents.

## 4. Then add metadata

This is where our earlier discussion becomes useful.

For each PDF, create metadata.

For example:

### v2

Python

Run

```
{
 "id": "support_policy_v2",
 "name": "Support Policy v2",
 "status": "deprecated",
 "effective_date": "2025-01-01",
 "document_type": "support_policy",
 "customer": None
}
```

### v3

Python

Run

```
{
 "id": "support_policy_v3",
 "name": "Support Policy v3",
 "status": "current",
 "effective_date": "2026-05-01",
 "document_type": "support_policy",
 "customer": None
}
```

### Northstar

Python

Run

```
{
 "id": "northstar_agreement",
 "name": "Northstar Enterprise Agreement",
 "status": "active",
 "effective_date": "2026-01-01",
 "document_type": "customer_agreement",
 "customer": "Northstar"
}
```

Notice something:

**We're not inventing business rules.**

We're simply organizing information already present in the PDFs.

For example, v2 explicitly says it is deprecated and superseded by v3.

02\_Support\_Policy\_v2\_DEPRECATED

---

## 5. Then organize the actual knowledge

Because the PDFs are small, I would initially organize them into **logical sections**, not blindly create tiny chunks.

For example:

```
Support Policy v3
|
├─ source_precedence
├─ severity_definitions
├─ response_targets
└─ escalation
```

Cancellation SOP:

```
Cancellation SOP
|
├─ cancellation_rules
├─ service_credit_rules
└─ uncertainty_rules
```

Product Guide:

```
Product Guide
|
├─ plan_capabilities
├─ shipment_status
├─ known_issue_KI208
├─ known_issue_KI211
└─ resolved_issue_KI176
```

Northstar:

```
Northstar Agreement
|
├─ support_terms
```

```
└─ cancellation_terms
└─ service_credit_terms
└─ account_contact
```

This is much more useful than blindly splitting every 300 characters.

---

## 6. Why logical sections are better here

Suppose the user asks:

"What is P1?"

We don't need the entire PDF.

We need:

```
Support Policy v3
→ severity_definitions
```

Suppose they ask:

"What happens when a pickup is late?"

We need:

```
Cancellation SOP
→ service_credit_rules
```

Suppose they ask:

"Why does the shipment still show BOOKED?"

We might need:

```
Product Guide
→ KI-211
```

This gives us **controlled context**.

---

## 7. Then build a simple retrieval function

For the first version, we can have:

Python

Run

```
search_documents(query)
```

Conceptually:

```
User question
 ↓
Search document sections
 ↓
Find relevant sections
 ↓
Return:
 content
 document
 metadata
```

For this small dataset, even a simple keyword/metadata-aware search can be enough for the initial prototype.

Then we can test whether retrieval is actually failing before introducing embeddings.

---

## 8. After that, decide whether we need vector search

This is the engineering decision I want you to understand.

Don't say:

"We're building RAG, therefore we need a vector database."

Instead ask:

**"What retrieval problem do we actually have?"**

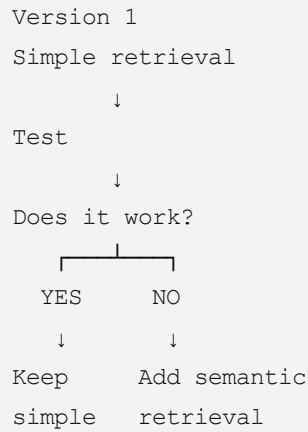
If our simple retrieval can reliably find:

```
P1
Northstar
cancellation
service credit
SLA
BOOKED
PICKED_UP
```

then we already have a functioning retrieval layer.

If natural-language questions aren't being matched reliably, **then** introduce embeddings/vector search.

That gives us:



That's much better engineering.

## 9. But there is something more important than retrieval

For this assessment, the hard problem isn't:

**"Can I find the PDF?"**

The hard problem is:

**"Can I determine which rule applies?"**

For example:

```
General SOP:
BOOKED > 30 min → ₹250

Northstar:
BOOKED before pickup → ₹0
```

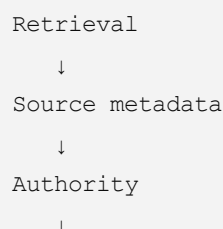
Both documents are relevant.

The system must determine that Northstar's agreement overrides the general rule.

03\_Cancellation\_and\_Service\_Cre...

05\_Northstar\_Logistics\_Enterpri...

So our architecture should prioritize:

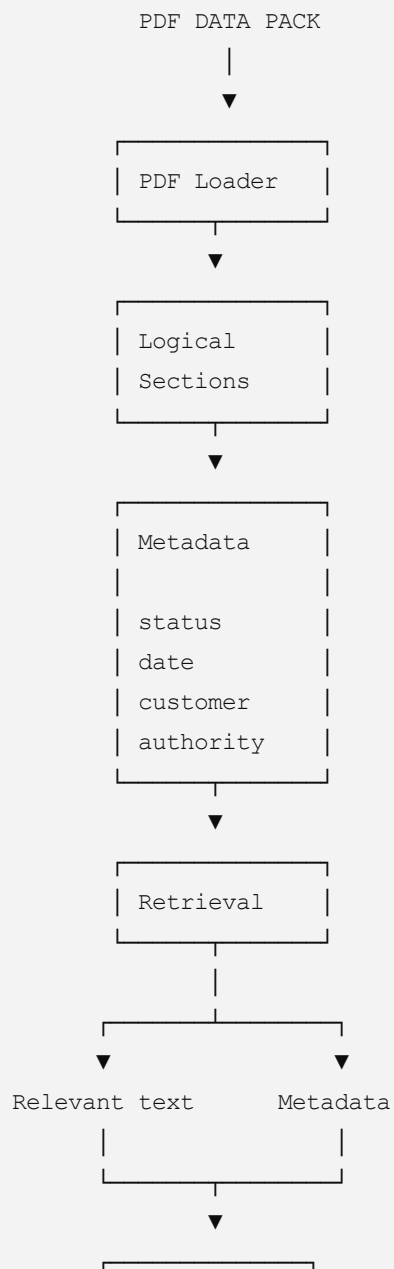


Customer context  
↓  
Conflict resolution  
↓  
Answer

not:

Vector DB  
↓  
LLM  
↓  
Answer

## 10. The architecture I recommend for our first version



|           |
|-----------|
| Agent     |
| Reasoning |

Then later:

```
Simple Retrieval
 ↓
Evaluation
 ↓
If insufficient
 ↓
Semantic Search / Vector DB
```

## 11. What I would NOT do right now

Because we only have a small dataset, I would **not** immediately build:

- ❌ complicated multi-agent system
- ❌ huge vector database
- ❌ sophisticated chunking framework
- ❌ reranking pipeline
- ❌ knowledge graph
- ❌ fine-tuned model
- ❌ MCP architecture just for appearance
- ❌ A2A
- ❌ complex memory system

Those may become useful later, but **we don't have evidence that we need them yet.**

## 12. One important exception: Excel

The PDFs are small, but the Excel workbook is different.

The assessment says it contains internal:

- account data
- order data
- ticket data

and should be used for both customer-facing and authorised internal workflows with appropriate access controls.

CalQuity AI Engineer — Job Desc...

So I would treat the two sources differently:



PDFs

↓

Knowledge / rules

↓

Document retrieval

Excel

↓

Operational records

↓

Structured queries

That's why our agent needs **two separate tools**:

```
search_documents()
query_operational_data()
```

And then the third:

```
create_escalation()
```

which satisfies the three tool categories required by the assessment.

CalQuity AI Engineer — Job Desc...

## ★ The correct first step

So when you say:

**"The PDF information is not so big. How can we create it first?"**

My answer is:

**Don't start with RAG.**

Start with a **document knowledge model**.

STEP 1

Load PDFs

↓

STEP 2

Extract text

↓

STEP 3

Identify logical sections

↓

STEP 4

Attach metadata

↓  
STEP 5  
Build simple retrieval  
↓  
STEP 6  
Test retrieval  
↓  
STEP 7  
Add conflict/source reasoning  
↓  
STEP 8  
Connect to agent

**Only after Step 6 do we decide whether embeddings/vector search are actually necessary.**

That follows the engineering principle we've been using:

**Don't choose the technology first. Understand the problem first, then introduce complexity only when the problem demands it.**

Sources

03\_Cancellation\_and\_Service\_Credit\_SOP\_v4.pdf  
explain about this

Yes. Let's understand 03\_Cancellation\_and\_Service\_Credit\_SOP\_v4.pdf **from zero**, slowly and practically.

This document is important because it tells the support team **what to do when a customer wants to cancel a shipment or wants compensation because a pickup was late.**

It has three main areas:

- 1. Can the shipment be cancelled?**
- 2. Does the customer qualify for a service credit?**
- 3. When should the agent stop and ask for human verification?**

The document is **CURRENT** and became effective **15 June 2026**.

03\_Cancellation\_and\_Service\_Cre...

# 1. First: What does "SOP" mean?

SOP means:

**Standard Operating Procedure**

In simple words:

**A step-by-step company rule for handling a particular situation.**

So this document is not a general support policy like v3.

Instead, it gives specific instructions for:

Cancellation  
+  
Service credits  
+  
Approval / uncertainty

Think of it like:

"When this particular situation happens, follow these rules."

---

## 2. Part 1 — Shipment Cancellation

This is the first major section.

The most important thing is:

**You cannot decide whether a shipment can be cancelled without knowing its current shipment status.**

The document gives different rules for:

DRAFT  
BOOKED  
PICKED\_UP  
DELIVERED

Let's understand each one.

---

## 3. DRAFT shipment

A **DRAFT** shipment has not really moved into the active shipment process yet.

The SOP says:

**May be cancelled with no fee.**

03\_Cancellation\_and\_Service\_Cre...

So:

```
Status = DRAFT
 ↓
Can cancel? YES
 ↓
Fee = ₹0
```

## Example

Customer says:

"I created a shipment but haven't finalized it. I want to cancel it."

If the data says:

```
Status = DRAFT
```

Then:

Cancel it without a fee.

---

## 4. BOOKED shipment

This is more interesting.

The SOP says a **BOOKED** shipment can be cancelled because the parcel hasn't been picked up yet. But the fee depends on **how long ago the shipment was booked**.

03\_Cancellation\_and\_Service\_Cre...

There are two situations.

### Situation A — Within 30 minutes

```
BOOKED
+
≤ 30 minutes since booking
 ↓
Cancellation allowed
```

↓  
Fee = ₹0

## Situation B — More than 30 minutes

BOOKED  
+  
> 30 minutes  
↓  
Normally ₹250 cancellation fee

But there is an important exception:

**Unless the customer's agreement explicitly waives the cancellation fee.**

03\_Cancellation\_and\_Service\_Cre...

This is where our **Northstar agreement** becomes important.

## 5. Northstar example

General SOP says:

BOOKED  
+  
more than 30 minutes  
↓  
₹250

But Northstar's agreement says:

Northstar can cancel any BOOKED shipment before pickup **without a cancellation fee**, regardless of how long ago it was booked.

05\_Northstar\_Logistics\_Enterpri...

So suppose:

Order = ORD-1001  
Customer = Northstar  
Status = BOOKED  
Booked = 2 hours ago

A basic system might say:

"₹250 fee because more than 30 minutes."

That would be **wrong**.

Our system needs to reason:

```
ORD-1001
 ↓
Northstar
 ↓
BOOKED
 ↓
2 hours old
 ↓
General SOP says ₹250
 ↓
Northstar agreement overrides it
 ↓
₹0
```

This is one of the best examples of why our AI needs **customer context + source precedence**.

---

## 6. PICKED\_UP shipment

Now things change.

The SOP says:

**Do not cancel.**

If the customer wants the parcel returned, use the:

**Return-to-origin workflow.**

03\_Cancellation\_and\_Service\_Cre...

So:

```
PICKED_UP
 ↓
Cannot simply cancel
 ↓
Customer wants parcel back?
 ↓
Return-to-origin process
```

**Why?**

Because once the carrier has picked up the parcel, the shipment has already moved into the delivery process.

The SOP doesn't say:

"Charge ₹250."

It says:

**Do not cancel. Use the return-to-origin workflow.**

That distinction matters.

---

## 7. DELIVERED shipment

This one is straightforward.

The SOP says:

**Cannot be cancelled.**

03\_Cancellation\_and\_Service\_Cre...

So:

DELIVERED

↓

Cancellation = NO

There is no cancellation procedure in this document for a delivered shipment.

We should not invent one.

---

## 8. Let's put the cancellation rules together

This is the mental model I want you to remember:

| Shipment status | Can cancel? | Fee / action  |
|-----------------|-------------|---------------|
| DRAFT           | Yes         | ₹0            |
| BOOKED ≤30 min  | Yes         | ₹0            |
| BOOKED >30 min  | Yes         | ₹250 normally |

| Shipment status                             | Can cancel? | Fee / action              |
|---------------------------------------------|-------------|---------------------------|
| <b>BOOKED &gt;30 min + agreement waiver</b> | Yes         | Agreement applies         |
| <b>PICKED_UP</b>                            | No          | Return-to-origin workflow |
| <b>DELIVERED</b>                            | No          | Cannot cancel             |

The first four rules are directly from the SOP, while the customer-specific override comes from the Northstar agreement.

03\_Cancellation\_and\_Service\_Cre...

05\_Northstar\_Logistics\_Enterpri...

## 9. Part 2 — Failed-pickup Service Credits

Now we move to a different problem.

Imagine a customer schedules a pickup:

Scheduled window  
10:00 - 11:00

But the carrier doesn't arrive on time.

The customer asks:

"Do I get compensation?"

ParcelPilot calls this compensation a:

### Service Credit

The SOP defines when the customer qualifies.

## 10. There are THREE conditions

Under the default rule, the customer gets a service credit only when **all three conditions** are satisfied.

### Condition 1

The pickup is:

**more than 2 hours past the end of the scheduled pickup window**



Notice the wording.

It's not:

"2 hours after the scheduled start."

It's:

**2 hours after the scheduled window ends.**

03\_Cancellation\_and\_Service\_Cre...

That's an important detail.

---

## Condition 2

The:

**carrier is at fault**

Meaning the delay wasn't simply because of ParcelPilot's customer.

---

## Condition 3

There is:

**no customer-caused issue.**

03\_Cancellation\_and\_Service\_Cre...

So the decision is:

```
Pickup > 2 hours late?
 AND
Carrier at fault?
 AND
Customer not at fault?
 ↓
YES
 ↓
Eligible for default service credit
```

If even one important condition is unknown, we shouldn't confidently promise a credit.

---

## 11. How much is the default credit?

The default credit is:

**The lower of ₹500 or 10% of the shipment fee.**

03\_Cancellation\_and\_Service\_Cre...

This means we calculate two numbers.

### Example

Shipment fee:

**₹4,000**

10%:

**₹400**

Compare:

₹500

₹400

Take the lower:

**₹400 credit**

---

### Another example

Shipment fee:

**₹10,000**

10%:

**₹1,000**

Compare:

₹500

₹1,000

Take the lower:

**₹500 credit**

So:

Credit =  $\min(\text{₹500}, 10\% \times \text{shipment fee})$

## 12. But customer agreements can change this

This is another major point.

The SOP says a signed customer agreement can replace:

- the delay threshold
- the credit amount
- the credit cap. 03\_Cancellation\_and\_Service\_Cre...

So again:

```
General SOP
 ↓
Default rule

Customer Agreement
 ↓
May override default
```

## 13. LumenWorks example

This is where the second customer agreement becomes useful.

General SOP:

```
>2 hours late
+
carrier at fault
+
customer not at fault
 ↓
lower of ₹500 or 10% fee
```

But LumenWorks has a special agreement:

```
>4 hours late
+
carrier at fault
+
customer not at fault
 ↓
₹300 fixed credit
```

And its agreement explicitly says this replaces the default threshold and credit amount.

06\_LumenWorks\_Service\_Agreement

So if LumenWorks' pickup is 3 hours late:

General SOP

→ Would potentially qualify

LumenWorks agreement

→ Needs >4 hours

Therefore:

→ Not eligible under LumenWorks's special rule

This is an excellent example of why our AI cannot just search for "service credit" and take the first answer.

---

## 14. Part 3 — Approval

There is another safety rule.

If an individual service credit is:

**above ₹1,000**

it requires:

**Manager approval.**

03\_Cancellation\_and\_Service\_Cre...

So:

Credit  $\leq$  ₹1,000

↓

Normal process

Credit  $>$  ₹1,000

↓

Manager approval required

Notice that this is about the **individual credit**, not the customer's monthly total.

---

## 15. Part 4 — What if we don't know enough?

This is probably the most important AI-agent lesson in this PDF.

Suppose the data says:

Pickup was late.

But we don't know:

Was the carrier responsible?  
Was the customer responsible?  
Exactly when did pickup occur?

The agent should **not say**:

"Yes, you are eligible for a credit."

The SOP explicitly says:

Do not promise a credit when carrier fault, pickup timing, or customer fault is unknown.

03\_Cancellation\_and\_Service\_Cre...

So the AI should instead say something like:

"The pickup appears to have been delayed, but I cannot confirm credit eligibility yet because the available information does not establish carrier fault."

That's a much safer answer.

---

## 16. What if two sources disagree?

The SOP also tells us what to do.

If data conflicts:

```
Conflict found
 ↓
Identify conflict
 ↓
Request verification
 ↓
Before state-changing action
```

The system should **not quietly choose one side**.

03\_Cancellation\_and\_Service\_Cre...

This connects directly to what we discussed earlier:

**Relevant information isn't automatically the correct information.**

## 17. Let's understand a complete example

Suppose we have:

```
Customer: LumenWorks
Shipment fee: ₹2,000
Scheduled pickup window:
10:00-11:00

Actual pickup:
15:30

Carrier fault:
YES

Customer fault:
NO
```

Let's reason step by step.

### Step 1 — When did the window end?

11:00.

### Step 2 — When was pickup?

15:30.

### Step 3 — How late?

$15:30 - 11:00 = 4.5 \text{ hours}$

So:

**4.5 hours late**

### Step 4 — Which customer?

LumenWorks.

### Step 5 — Does LumenWorks have a special agreement?

Yes.

### Step 6 — What does the agreement say?

More than 4 hours + carrier fault + customer not at fault:

**₹300 fixed credit.**

06\_LumenWorks\_Service\_Agreement

## Step 7 — Final result

**₹300 service credit**, assuming the relevant facts are verified.

Notice what happened:

```
Operational data
 +
Customer agreement
 +
 SOP
 ↓
Final decision
```

That's exactly the type of **multi-source reasoning** our agent needs.

---

## 18. What this means for our AI architecture

This PDF tells us we need structured calculations, not just document search.

For example:

```
check_cancellation_eligibility()
```

could consider:

```
customer
shipment_status
booking_time
current_time
customer_agreement
```

And:

```
calculate_service_credit()
```

could consider:

```
customer
scheduled_window
actual_pickup
carrier_fault
customer_fault
shipment_fee
customer_agreement
```

So the AI shouldn't calculate everything itself from raw text.

A safer design is:

```
Agent
 ↓
Retrieve relevant rules
 ↓
Get operational data
 ↓
Call deterministic calculation/tool
 ↓
Reason over result
 ↓
Answer
```

---

## 19. The most important lessons from this PDF

### Lesson 1 — Status matters

You cannot answer a cancellation question without knowing:

**DRAFT / BOOKED / PICKED\_UP / DELIVERED**

---

### Lesson 2 — Time matters

For BOOKED shipments:

**30 minutes** changes the cancellation fee.

For failed pickups:

**2 hours after the end of the scheduled window** matters.

---

### Lesson 3 — Customer agreements matter

General SOP = default.

Customer agreement = possible override.

Northstar and LumenWorks demonstrate this clearly.

06\_LumenWorks\_Service\_Agreement

05\_Northstar\_Logistics\_Enterpri...

---



## Lesson 4 — Don't guess when facts are missing

Unknown carrier fault?

Unknown customer fault?

Unknown pickup timing?

**Don't promise a credit.**

03\_Cancellation\_and\_Service\_Cre...

## Lesson 5 — Don't hide conflicts

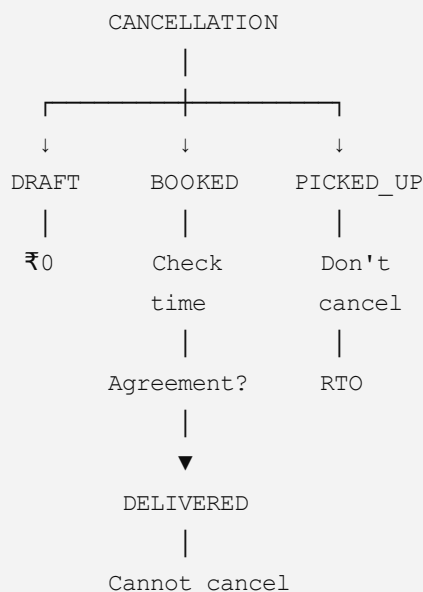
If the data disagrees:

**identify the conflict → verify → then act.**

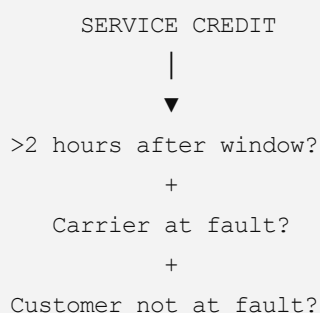
03\_Cancellation\_and\_Service\_Cre...

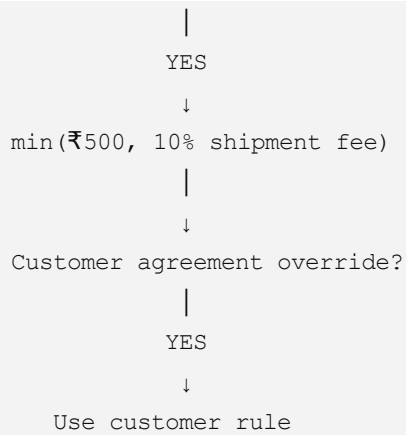
## ★ The simplest mental model

Remember this PDF as:

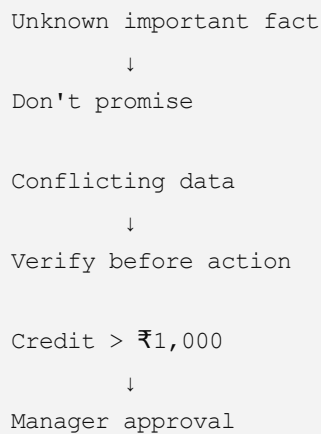


And:





Then one final safety layer:



If you understand those three flows, you understand the core of this PDF. And this is also why the PDF is more important to our agent than simply putting all its text into a vector database: the agent needs to turn these rules into **structured decision logic + tool calls**, while using the documents as authoritative evidence.

Sources

03\_Cancellation\_and\_Service\_Credit\_SOP\_v4.pdf a clear notes of it

03\_Cancellation\_and\_Service\_Credit\_SOP\_v4.pdf —

## Clear Notes

**Document type:** Standard Operating Procedure (SOP)

**Status:** CURRENT

**Effective date:** 15 June 2026

03\_Cancellation\_and\_Service\_Cre...

## 1. Purpose of the document

This document tells ParcelPilot support staff **what to do in two situations**:

1. A customer wants to **cancel a shipment**.
2. A customer wants a **service credit because a pickup was delayed**.

It also explains what to do when there is **missing or conflicting information**.

03\_Cancellation\_and\_Service\_Cre...

---

## PART A — CANCELLATION RULES

### 2. First thing to check: shipment status

Before deciding whether a shipment can be cancelled, check its status.

There are four important statuses:

DRAFT  
BOOKED  
PICKED\_UP  
DELIVERED

The cancellation rule is different for each one.

---

### 3. DRAFT

#### Rule

A DRAFT shipment:

- **Can be cancelled**
- **No cancellation fee**

03\_Cancellation\_and\_Service\_Cre...

#### Remember

**DRAFT → Cancel → ₹0**

---

### 4. BOOKED

A BOOKED shipment can normally be cancelled because it has not yet been picked up.

But the fee depends on how long ago it was booked.

03\_Cancellation\_and\_Service\_Cre...

## Within 30 minutes

BOOKED  
+  
≤ 30 minutes  
↓  
Cancel allowed  
↓  
₹0 fee

## More than 30 minutes

BOOKED  
+  
> 30 minutes  
↓  
Cancel allowed  
↓  
₹250 fee

**But:** a customer's signed agreement can waive this fee.

03\_Cancellation\_and\_Service\_Cre...

## 5. PICKED\_UP

Once the shipment has been picked up:

**Do not cancel it using the normal cancellation process.**

If the customer wants the parcel returned, use the:

**Return-to-Origin (RTO) workflow.**

03\_Cancellation\_and\_Service\_Cre...

**Remember**

**PICKED\_UP → Don't cancel → RTO**

## 6. DELIVERED

A DELIVERED shipment:

**Cannot be cancelled.**

03\_Cancellation\_and\_Service\_Cre...

**Remember**

**DELIVERED → No cancellation**

# Cancellation Cheat Sheet

| Status                    | Cancellation | Fee / Action           |
|---------------------------|--------------|------------------------|
| DRAFT                     | ✓ Yes        | ₹0                     |
| BOOKED ≤30 min            | ✓ Yes        | ₹0                     |
| BOOKED >30 min            | ✓ Yes        | ₹250 normally          |
| BOOKED + agreement waiver | ✓ Yes        | Agreement rule applies |
| PICKED_UP                 | ✗ No         | Use RTO workflow       |
| DELIVERED                 | ✗ No         | Cannot cancel          |

03\_Cancellation\_and\_Service\_Cre...

## PART B — FAILED-PICKUP SERVICE CREDITS

### 7. What is a service credit?

A service credit is compensation given to a customer when a pickup has been sufficiently delayed **and the required conditions are met**.

It is **not automatically given just because a pickup was late**.

### 8. Three conditions must be checked

Under the default SOP, all three conditions must be satisfied:

#### Condition 1 — Pickup delay

The pickup is:

**More than 2 hours late after the end of the scheduled pickup window.**

#### Condition 2 — Carrier responsibility

The:

**Carrier is at fault.**

## Condition 3 — Customer responsibility

There is:

**No customer-caused issue.**

03\_Cancellation\_and\_Service\_Cre...

So remember:

```
>2 hours late
+
Carrier at fault
+
Customer NOT at fault
↓
Potential service credit
```

## 9. How much is the default credit?

The default credit is the **lower of**:

- **₹500**
- **10% of the shipment fee**

03\_Cancellation\_and\_Service\_Cre...

### Example

Shipment fee = ₹4,000

10% = ₹400

Compare:

₹500

₹400

Take the lower amount:

**₹400 credit**

### Formula

Service Credit =  $\min(\text{₹500}, 10\% \times \text{Shipment Fee})$

## 10. Customer agreements can override the default

This is very important.

A signed customer agreement can change:

- the delay threshold
- the credit amount
- the credit cap. 03\_Cancellation\_and\_Service\_Cre...

So don't automatically apply the general SOP.

Always ask:

**Does this customer have a special agreement?**

---

## 11. Example — LumenWorks

The general SOP says:

```
>2 hours late
+
Carrier at fault
+
Customer not at fault
→
Lower of ₹500 or 10% of shipment fee
```

But LumenWorks has a special agreement.

For LumenWorks:

```
>4 hours late
+
Carrier at fault
+
Customer not at fault
→
₹300 fixed credit
```

The LumenWorks agreement specifically replaces the default threshold and credit amount.

06\_LumenWorks\_Service\_Agreement

### Important

If LumenWorks is **3 hours late**, don't use the general 2-hour rule.

Use the LumenWorks agreement:

3 hours < 4 hours → does not meet the special threshold.

---

## PART C — SAFETY / UNCERTAINTY

### 12. Don't promise a credit when important information is missing

Suppose we know:

"The pickup was late."

But we don't know:

- whether the carrier was responsible
- whether the customer caused the problem
- the exact pickup timing

Then the agent should **not promise the customer a service credit**.

03\_Cancellation\_and\_Service\_Cre...

Instead:

**Verify the missing information first.**

### Remember

Missing important fact

↓

Don't promise credit

↓

Verify

---

### 13. What if the data conflicts?

Suppose one system says:

Pickup happened at 10:00.

Another source says:

Pickup happened at 14:00.

We have a conflict.

The SOP says the conflict should be:



1. Identified
2. Verified
3. Resolved before taking a state-changing action.

03\_Cancellation\_and\_Service\_Cre...

### Don't do this

"I'll just choose whichever looks more likely."

### Do this

"The available records conflict. I need verification before proceeding."

## 14. Manager approval

If an individual service credit is:

**Above ₹1,000**

it requires **manager approval**.

03\_Cancellation\_and\_Service\_Cre...

So:

Credit  $\leq$  ₹1,000

↓

Normal process

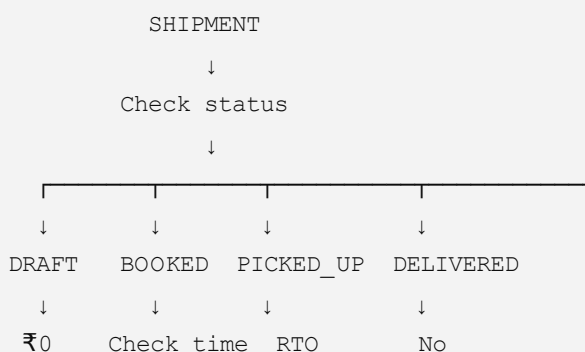
Credit  $>$  ₹1,000

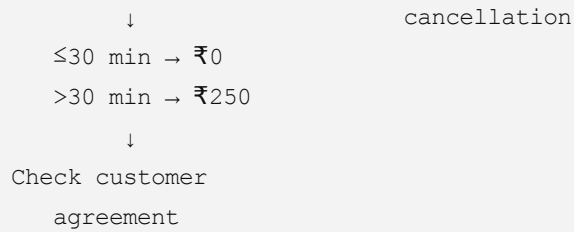
↓

Manager approval

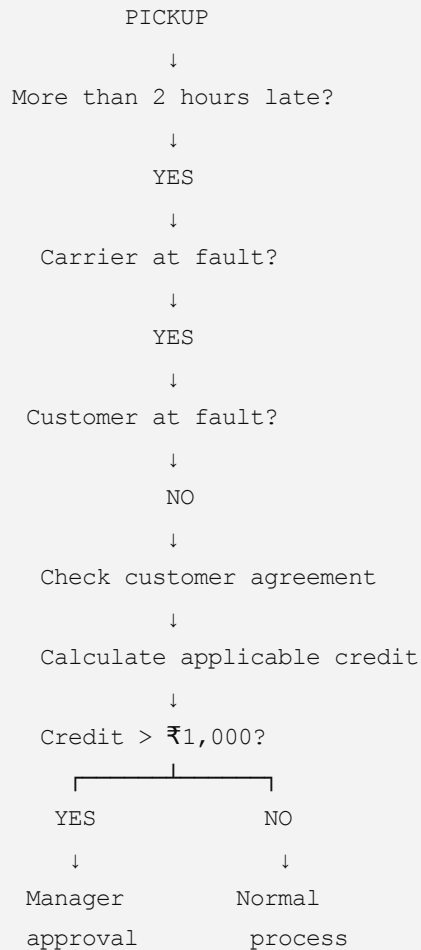
## The complete decision flow

### Cancellation





## Service Credit



## ★ What you should remember

If you only write **10 lines on your wall**, write these:

1. DRAFT → cancel, ₹0.
2. BOOKED ≤30 min → cancel, ₹0.
3. BOOKED >30 min → ₹250 normally.
4. Customer agreement can override cancellation fee.
5. PICKED\_UP → don't cancel; use RTO.

6. DELIVERED → cannot cancel.
7. Service credit needs >2 hours after pickup-window end.
8. Carrier must be at fault + customer must NOT be at fault.
9. Default credit = lower of ₹500 or 10% of shipment fee.
10. Missing/conflicting information → verify; don't promise.

And the **AI-engineering lesson** from this document is:

**Never make a cancellation or compensation decision from the PDF alone.  
Combine the rule from the SOP with the customer's agreement and the actual operational data.**

That is the core reason this document will eventually connect to both our **document retrieval system** and **Excel/data tools**.

04\_Product\_Operations\_Guide\_and\_Known\_Issues.pdf  
explain about this

Absolutely. Let's understand 04\_Product\_Operations\_Guide\_and\_Known\_Issues.pdf from the beginning, in simple language.

This document is different from the first three.

- **Support Policy v3** → tells us **how serious a support problem is and how quickly to respond**.
- **Support Policy v2** → old version of those rules.
- **Cancellation & Service Credit SOP** → tells us **what to do about cancellations and delayed pickups**.
- **Product Operations Guide** → tells us **how ParcelPilot's product actually works and what known technical problems exist**.

The document is **current** and was updated **14 August 2026**.

04\_Product\_Operations\_Guide\_and...

# 1. What is this document?

Think of this PDF as a **user/operations manual for ParcelPilot**.

It answers questions like:

"Can this customer use bulk upload?"

"What does BOOKED mean?"

"Why does my shipment still show BOOKED even though the carrier picked it up?"

"Why is my large CSV upload failing?"

So its purpose is:

```
How does ParcelPilot work?
+
What problems are already known?
+
What should support tell the customer?
```

---

## 2. Part 1 — Product capabilities

The first thing the document tells us is about **Bulk Upload**.

### What is Bulk Upload?

Instead of creating shipments one at a time, a customer can upload a CSV file containing many shipments.

For example:

```
shipment 1
shipment 2
shipment 3
...
shipment 1000
```

Instead of manually creating all 1,000 shipments, the customer uploads them together.

---

## 3. Which plans support Bulk Upload?

The document says:

| Plan       | Bulk Upload                                                                           |
|------------|---------------------------------------------------------------------------------------|
| Standard   |  No  |
| Growth     |  Yes |
| Enterprise |  Yes |

The supported CSV limit is:

**5,000 rows.** 04\_Product\_Operations\_Guide\_and...

So if a Standard customer asks:

"Why can't I use Bulk Upload?"

The answer isn't necessarily that the system is broken.

It may simply be:

**Bulk Upload isn't included in the Standard plan.**

That's an important distinction.

---

## 4. Important: Official limit vs known problem

Now we get to one of the most interesting parts of this document.

The official supported limit is:

**5,000 rows.**

But there is a known problem.

That's **KI-208**.

---

## 5. KI-208 — Large CSV upload problem

### What is KI-208?

Some **Growth and Enterprise** customers experience intermittent failures when uploading CSV files above approximately:

## 3,000 rows

This can happen even though the official supported limit is:

**5,000 rows.** 04\_Product\_Operations\_Guide\_and...

This creates an important distinction:

```
Officially supported
 ↓
5,000 rows

Known practical issue
 ↓
Problems may occur above ~3,000 rows
```

These are not actually contradictory.

The product officially supports up to 5,000 rows, but there is a known reliability problem around larger uploads.

---

## 6. What is the workaround?

The document says:

Split the upload into files below 3,000 rows. 04\_Product\_Operations\_Guide\_and...

For example, instead of:

```
6,000 rows
 ↓
ONE FILE
 ↓
❌ likely to encounter KI-208
```

Do:

```
File 1 → 2,500 rows
File 2 → 2,500 rows
File 3 → 1,000 rows
```

This works around the known issue.

---

## 7. Does KI-208 affect individual shipment creation?

No.

The document says:

Individual shipment creation is unaffected.

04\_Product\_Operations\_Guide\_and...

So:

Bulk Upload

↓

Potential problem above ~3,000 rows

Individual shipment

↓

Not affected

This is useful because the support agent can offer a workaround rather than simply saying:

"The system is broken."

---

## 8. KI-211 — Delayed pickup status

This is another important known issue.

It involves:

**SwiftShip pickup webhooks.**

Let's first understand what a webhook means in simple terms.

A webhook is basically an automatic message from one system to another.

Here:

SwiftShip

↓

"Pickup happened"

↓

ParcelPilot

ParcelPilot uses that information to update the shipment status.

---

## 9. What is the KI-211 problem?

The document says:

SwiftShip pickup webhooks can be delayed by up to **20 minutes**.

04\_Product\_Operations\_Guide\_and...

So imagine:

```
10:00
Carrier actually picks up parcel
 ↓
10:00-10:20
Webhook delayed
 ↓
ParcelPilot may still show:
BOOKED
```

This means:

**BOOKED does not always immediately mean the carrier hasn't picked up the parcel.**

That's a very important operational detail.

---

## 10. Why is this important?

Imagine the customer says:

"The carrier picked up my parcel, but ParcelPilot still says BOOKED."

A careless support agent might say:

"The carrier hasn't picked it up."

That could be wrong.

The Product Operations Guide tells us that SwiftShip webhooks can be delayed by up to 20 minutes.

04\_Product\_Operations\_Guide\_and...

So the correct approach is:

```
ParcelPilot says BOOKED
 ↓
Is carrier = SwiftShip?
 ↓
YES
 ↓
Check carrier status
OR
```



allow the known 20-minute delay

↓

Then determine actual pickup status

The document explicitly says support should **verify carrier status or wait through the known delay window before telling the customer pickup did not happen.**

04\_Product\_Operations\_Guide\_and...

---

## 11. Why this matters for cancellation

Now connect this PDF to the previous SOP.

Remember:

BOOKED

→ cancellation may be possible

PICKED\_UP

→ don't cancel normally

Now imagine:

ParcelPilot says:

BOOKED

But:

SwiftShip actually picked it up 10 minutes ago

If our AI blindly trusts the database status and cancels the shipment, it could make a **wrong operational decision.**

This is why the Product Guide is not just documentation.

It provides **context needed to correctly interpret operational data.**

---

## 12. KI-176 — Resolved issue

The document also contains:

### KI-176

This was an address-validation issue.

But it was **resolved on 18 July 2026.**

04\_Product\_Operations\_Guide\_and...

That means:

```
KI-176
 ↓
Old problem
 ↓
Resolved
 ↓
Don't automatically blame current incidents on it
```

This is another example of why **status and dates matter**.

---

## 13. Why keep a resolved issue?

You might ask:

"If the problem is fixed, why keep it?"

For historical/reference purposes.

Suppose an old support ticket from June says:

"Address validation is failing because of KI-176."

That could make sense historically.

But if a customer reports an address-validation problem in August, the AI shouldn't automatically say:

"It's KI-176."

Because the issue was resolved on 18 July.

Instead:

Investigate the current evidence.

Again:

**Historical information ≠ current explanation.**

---

## 14. The three issues in this document

You can remember the document like this:

| Item   | Problem                       | Important fact                       | Action                                    |
|--------|-------------------------------|--------------------------------------|-------------------------------------------|
| KI-208 | Large CSV uploads             | Failures may occur above ~3,000 rows | Split files below 3,000                   |
| KI-211 | SwiftShip pickup status delay | Webhooks can be delayed up to 20 min | Verify carrier status / wait              |
| KI-176 | Address validation            | Resolved 18 July 2026                | Don't assume it explains current problems |

04\_Product\_Operations\_Guide\_and...

## 15. The deeper lesson: operational data can be misleading

This PDF teaches us something very important for our AI agent.

Suppose Excel says:

```
Order:
Status = BOOKED
```

A naive AI might conclude:

"The parcel hasn't been picked up."

But the Product Guide tells us:

For SwiftShip, the pickup webhook can be delayed by up to 20 minutes.

04\_Product\_Operations\_Guide\_and...

So the AI needs to combine:

```
Excel
+
Product Guide
+
Customer context
+
Current time
↓
Correct interpretation
```

This is exactly why we need **multiple knowledge sources**.

---

## 16. Another example: Bulk Upload

Customer says:

"My 4,000-row CSV upload keeps failing."

The agent should not immediately say:

"ParcelPilot only supports 3,000 rows."

That would be wrong.

The document says:

```
Official limit = 5,000
Known issue = intermittent failures above ~3,000
```

So the better answer is:

"Your 4,000-row file is within the official 5,000-row limit, but there is a known issue affecting larger uploads above approximately 3,000 rows. Try splitting the file into smaller batches below 3,000 rows."

That is a much more accurate support answer.

---

## 17. How this PDF connects to the other documents

This is where the full system starts becoming clear.

### Support Policy

Answers:

**How serious is the problem?**

### Cancellation SOP

Answers:

**What should we do about cancellation/credits?**

### Product Guide

Answers:

How does the product actually behave?

## Customer Agreement

Answers:

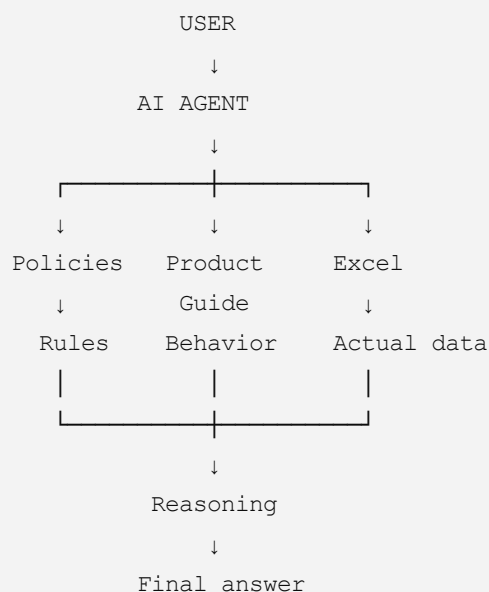
Does this customer have special rules?

## Excel

Answers:

What actually happened to this customer/order/ticket?

So we might have:



## 18. What should our AI do with this PDF?

This document suggests that our agent needs to recognize **known issues as explanations for observed behavior**.

For example:

User:

"Why is my SwiftShip order still BOOKED?"

Agent should reason:

1. Find order
2. Check carrier
3. Check status
4. Find KI-211
5. Check whether delay is within 20 minutes
6. If yes → explain known webhook delay
7. If beyond expected window → verify/escalate

That's much better than simply retrieving KI-211 and saying:

"There is a 20-minute delay."

The agent must connect the **document rule to the actual operational data**.

---

## 19. What you should remember

If you're making notes, write these:

04\_Product\_Operations\_Guide

CURRENT

Updated: 14 Aug 2026

### 1. BULK UPLOAD

Growth + Enterprise → available

Standard → not available

Official limit → 5,000 rows

### 2. KI-208

> ~3,000 rows → intermittent upload failures

Workaround → split files below 3,000

Individual shipment creation → unaffected

### 3. SHIPMENT STATUS

BOOKED → shipment created, pickup not yet confirmed

PICKED\_UP → carrier confirmed pickup

### 4. KI-211

SwiftShip webhook can be delayed up to 20 min

→ BOOKED may temporarily remain even after pickup

→ verify carrier status / wait

### 5. KI-176

Address validation issue

→ resolved 18 July 2026

→ don't automatically use it to explain current issues

## ★ The most important idea

Don't think of this PDF as just:

"A document containing product information."

Think of it as:

"A guide that helps the AI interpret what it sees in the operational data correctly."

For example:

Excel says:

BOOKED

Product Guide says:

SwiftShip webhook can be delayed 20 min

Therefore:

Don't immediately conclude "not picked up."

That is the real value of this document in our AI system.

**The PDF gives us the rules for interpreting reality; the Excel gives us the current snapshot of that reality.**

what is mean by bulk upload is the data set not understood  
this is just a parcel taker and giver right then in middle why  
we got data set csv and so on

Yes — your confusion is reasonable. **The assessment is not describing ParcelPilot as simply “a person gives a parcel to another person.”** It is describing a **shipping/logistics software platform** that businesses use to manage many shipments.

Let's separate the real-world business from the data pack.

# 1. What is ParcelPilot?

Think of ParcelPilot as **software used by a business that sends parcels**.

For example, imagine an online clothing company:

```
Customer orders a shirt
 ↓
Clothing company needs to ship it
 ↓
ParcelPilot helps the company create/manage the shipment
 ↓
A carrier picks up the parcel
 ↓
Carrier delivers it
```

So ParcelPilot is **not necessarily the courier itself**.

It is more like the **software layer that helps businesses manage their shipments and interact with carriers**.

The assessment describes the system as a support platform where an AI assistant needs to work with customer accounts, orders, tickets, policies, agreements, and operational data. The supplied workbook contains account, order, and ticket data. CalQuity AI Engineer — Job Desc...

---

## 2. Then what is a "shipment"?

A shipment is basically:

**One parcel/order that a business is sending through a carrier.**

For example:

```
ORD-1001

Customer: Northstar Logistics
Shipment: Parcel containing products
Carrier: SwiftShip
Status: BOOKED
```

The business needs software to keep track of things like:

- Who is sending it?
- Where is it going?
- Which carrier is handling it?
- Has it been booked?



- Has it been picked up?
- Has it been delivered?
- Was there a problem?

That's where the dataset comes in.

---

### 3. Why do we have Excel data?

This is the part that was confusing you.

The PDFs tell us **the rules**.

The Excel tells us **what is actually happening to particular customers, orders, and support tickets**.

Think:

#### PDFs = Rulebook

"What should happen?"

#### Excel = Reality

"What actually happened?"

That's the cleanest way to understand the whole assessment.

---

### 4. Example

Suppose the PDF says:

A BOOKED shipment can be cancelled.

That's a **general rule**.

But the AI receives:

"Can I cancel ORD-1001?"

How can it answer?

It needs to look up:

```
ORD-1001
 ↓
Who owns it?
 ↓
Northstar
 ↓
What is its status?
 ↓
BOOKED
```

Now it combines:

```
Excel:
ORD-1001 is BOOKED

PDF:
BOOKED shipment can be cancelled

Northstar Agreement:
Northstar can cancel BOOKED shipments without a cancellation fee
```

Then it can answer correctly.

---

## 5. So why CSV?

This is where **Bulk Upload** comes in.

Imagine a company doesn't have 5 shipments.

Imagine it has:

**5,000 shipments.**

Creating them one by one would be ridiculous.

Instead, the company can prepare a spreadsheet/CSV:

```
order_id,customer,address,carrier,...
ORD-001,...
ORD-002,...
ORD-003,...
...
ORD-5000,...
```

Then upload the file.

That's **Bulk Upload**.

**Bulk = many things at once.**

So:

```
Create shipment 1
Create shipment 2
Create shipment 3
...
Create shipment 5000
```

can become:

```
Upload one CSV containing many shipment records
```

The Product Operations Guide says Bulk Upload is available to **Growth and Enterprise** customers, with an official CSV limit of **5,000 rows**.

04\_Product\_Operations\_Guide\_and...

## 6. Why does a logistics company need this?

Imagine an e-commerce company receives:

```
Monday:
2,000 orders

Tuesday:
3,500 orders

Wednesday:
1,800 orders
```

They need to create thousands of shipments.

They don't want an employee sitting there doing:

```
Enter shipment
→ Save

Enter shipment
→ Save

Enter shipment
→ Save

... × 5,000
```

Instead:

```
E-commerce system
 ↓
CSV export
 ↓
ParcelPilot Bulk Upload
 ↓
Thousands of shipment records
```

That's why CSV exists.

---

## 7. But here's the important distinction

You asked:

"Why do we have a dataset CSV in the assessment?"

The **customer's CSV upload** and the **assessment Excel dataset** are related concepts, but they are **not the same thing**.

### Product CSV

This represents a customer's operational use of ParcelPilot.

For example:

```
Customer has 4,000 shipments
 ↓
Uploads CSV
 ↓
ParcelPilot processes them
```

### Assessment Excel

This is **test data supplied to us by CalQuity**.

It gives our AI something realistic to work with:

```
Accounts
Orders
Tickets
```

The assessment specifically says the workbook is the **source of truth for structured-data questions**.

CalQuity AI Engineer — Job Desc...

So don't think:

"The Excel is a customer uploading a CSV."

Think:

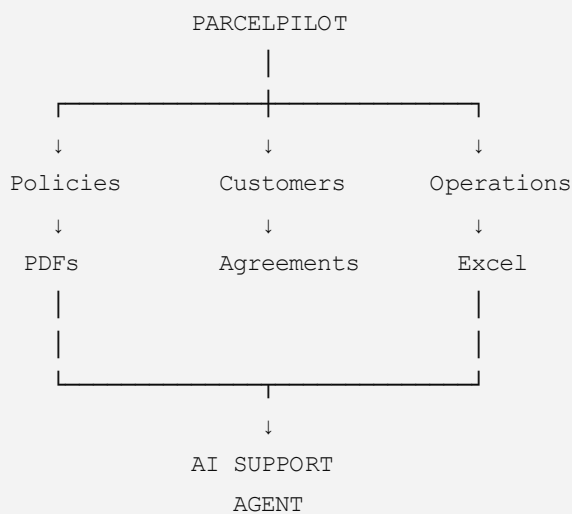
"CalQuity gives us a simulated ParcelPilot database so we can build and test the support agent."

## 8. The whole assessment is basically a simulated company

This is the key realization.

CalQuity has created a **small fictional ParcelPilot environment** for us.

They give us:



The AI is supposed to act like a **support employee** inside this fictional company.

## 9. Why does the AI need all of this?

Because a real support question isn't always answered by one source.

Imagine the user says:

"Can I cancel my shipment ORD-1001?"

The agent needs to investigate:

## Step 1 — Excel

What is ORD-1001 ?

Customer?  
Status?  
Carrier?  
Booking time?

## Step 2 — SOP

What does the normal cancellation rule say?

## Step 3 — Customer agreement

Does this customer have special cancellation terms?

## Step 4 — Decision

Combine everything.

That's much closer to how a real support employee would work.

---

# 10. And this explains why the assessment has so many documents

At first it looks strange:

"Why do I have a Support Policy, another old Support Policy, an SOP, Product Guide, two agreements, and an Excel?"

But each one represents a different type of information.

| Source               | What it tells the AI                             |
|----------------------|--------------------------------------------------|
| Support Policy v3    | How serious is the problem? How quickly respond? |
| Support Policy v2    | Historical/old rules                             |
| Cancellation SOP     | What to do for cancellation/credits              |
| Product Guide        | How the product behaves / known issues           |
| Northstar Agreement  | Special rules for Northstar                      |
| LumenWorks Agreement | Special rules for LumenWorks                     |

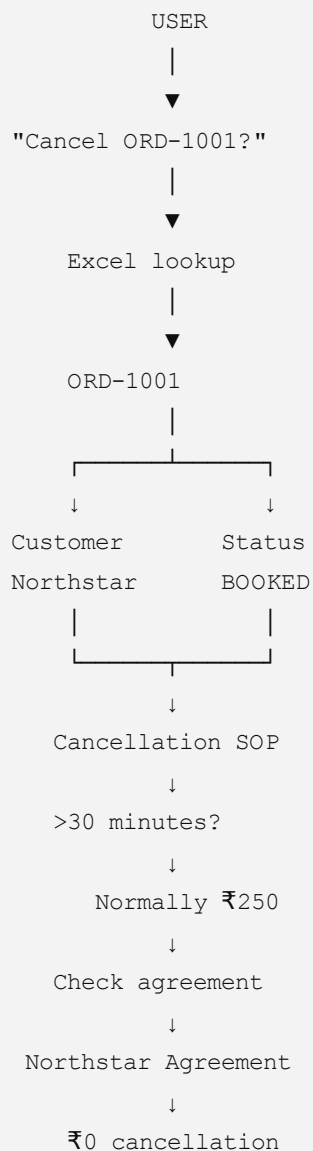
## 11. One example shows the entire system

Suppose the user asks:

**"Why can't I cancel ORD-1001 without paying a fee?"**

Our agent shouldn't immediately answer.

It should investigate:



Now the AI can say:

"ORD-1001 is a BOOKED Northstar shipment. Although the general cancellation SOP would normally charge ₹250 for a BOOKED shipment cancelled after 30 minutes, Northstar's agreement waives the cancellation fee."

That's a **real AI-agent workflow**, not just a chatbot answering from a PDF.

## 12. So your mental model should be this

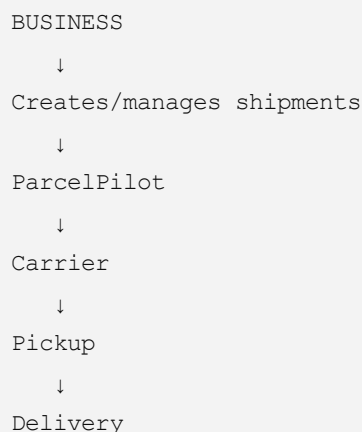
Don't think:

**ParcelPilot = parcel giver + parcel taker.**

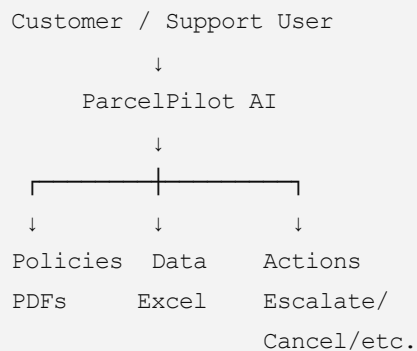
Think:

**ParcelPilot = software platform used by businesses to manage their parcel shipments and the operational/support processes around them.**

Then:



And our assessment AI sits on the **support side**:



**The simplest sentence:**



The PDFs tell our AI what the rules are; the Excel tells our AI what is happening; the AI combines both to help the customer or support employee make the correct decision.

That's why we have both **CSV/product data concepts** and the **assessment dataset**. They serve different purposes.

Sources

we have less rows mean less orders or less products  
transport means in one order there is 10 pants if for 10 pants  
take 10 rows or 1 row

Exactly — this is an important distinction. **“Rows” does not automatically mean “number of products.”** It depends on what **one row represents** in that particular dataset.

For the ParcelPilot assessment, the Product Operations Guide says the Bulk Upload limit is **5,000 rows**, but it does **not specify in that PDF whether one row represents one product, one parcel, one order, or something else**. So we should not assume that from the PDF alone.

04\_Product\_Operations\_Guide\_and...

### Think of it like this

Suppose a customer has one order:

**Order ORD-1001:** 10 pants going to one customer.

If the system models **one shipment/order per row**, it could be:

| Order ID | Product | Quantity | Destination |
|----------|---------|----------|-------------|
| ORD-1001 | Pants   | 10       | Hyderabad   |

That's **1 row**.

It would **not** normally need 10 rows just because there are 10 pants.

But another system could model each item separately:

| Order ID | Item    | Quantity |
|----------|---------|----------|
| ORD-1001 | Pant 1  | 1        |
| ORD-1001 | Pant 2  | 1        |
| ...      | ...     | ...      |
| ORD-1001 | Pant 10 | 1        |

That's **10 rows**.

**The key question is:**

**What does one row represent in ParcelPilot's actual data model?**

We shouldn't guess this.

For our assessment, we need to inspect the actual `ParcelPilot_Assessment_Data.xlsx` columns and README. The assessment says the workbook contains **account, order, and ticket data**, so the exact row meaning should come from that workbook's structure.

CalQuity AI Engineer — Job Desc...

So remember:

**5,000 rows ≠ necessarily 5,000 products.**

It means **5,000 records of whatever the Bulk Upload format defines as a record**.

And this is exactly why, before we implement the Bulk Upload/operational-data logic, we should inspect the actual dataset schema rather than inventing assumptions.

then if so why customers needs 3000 rows actually what did they do with that they write it in one row right

Yes — **your logic is correct up to one point**: if one row represents one shipment/order record, then **10 pants in that shipment could be represented in one row** with `quantity = 10`.

But there is a gap: **we don't yet know what ParcelPilot's bulk-upload row represents.** The PDF only tells us that the official CSV limit is 5,000 rows and that uploads above ~3,000 rows can intermittently fail. It does **not** define the exact CSV schema.

04\_Product\_Operations\_Guide\_and...

## So why might a customer have 3,000 rows?

Imagine a business has **3,000 separate shipments** to create.

For example:

```
Row 1 → Shipment ORD-001 → 1 customer → 1 destination
Row 2 → Shipment ORD-002 → 1 customer → 1 destination
Row 3 → Shipment ORD-003 → 1 customer → 1 destination
...
Row 3000 → Shipment ORD-3000
```

They could prepare those 3,000 shipment records in a CSV and upload them together.

So **3,000 rows could mean 3,000 shipment records**, not 3,000 pants.

## Your pants example

Suppose:

Order ORD-1001 contains 10 pants, all going to the same destination.

If the upload format represents **one shipment per row**, conceptually:

| Shipment ID | Product | Quantity | Destination |
|-------------|---------|----------|-------------|
| ORD-1001    | Pants   | 10       | Hyderabad   |

That's **1 shipment row**, even though there are 10 pants.

But suppose the business has:

3,000 different customers/orders that need shipping.

Then:

**3,000 shipment records → potentially 3,000 rows.**

---

## Why does ParcelPilot have Bulk Upload?

Because businesses can have **thousands of shipments**, especially e-commerce/logistics businesses.

Instead of doing:

```
Create shipment
→ save
→ create shipment
→ save
→ ...
→ repeat thousands of times
```

they can prepare a CSV and upload many records at once.

The Product Guide says:

- **Growth + Enterprise:** Bulk Upload available
- **Standard:** Bulk Upload unavailable
- Official limit: **5,000 rows**
- Known issue: uploads above approximately **3,000 rows** can intermittently fail
- Workaround: split into files below 3,000 rows.

04\_Product\_Operations\_Guide\_and...

## One correction to our earlier discussion

We should **not yet say "one row = one order" as a fact.**

That's an assumption until we inspect the actual bulk-upload schema or assessment workbook.

The correct engineering question is:

**What does one Bulk Upload row represent in ParcelPilot?**

Once we inspect that schema, we'll know whether:

```
1 row = 1 shipment
```

or

```
1 row = 1 order
```

or something else.

**Don't build the system around that assumption yet.** That's exactly the kind of small assumption that can later cause a wrong implementation.

05\_Northstar\_Logistics\_Enterprise\_Agreement.pdf  
explain about this

Absolutely. Let's understand 05\_Northstar\_Logistics\_Enterprise\_Agreement.pdf from zero.

This document is very important because it is **not a general ParcelPilot policy**. It is a **special contract between ParcelPilot and one particular customer: Northstar Logistics**.

The main idea is:

**Northstar has some rules that are different from the normal ParcelPilot rules.**

And because this is a **signed customer agreement**, those special rules can override the general support policy and SOP where the agreement says they do.

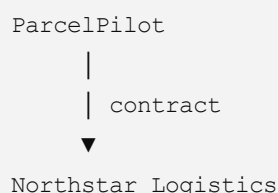
## 1. What is Northstar?

Northstar Logistics is one of ParcelPilot's customers.

The document identifies:

- **Account:** ACCT-001
- **Customer:** Northstar Logistics
- **Plan:** Enterprise
- **Contract period:** 1 January 2026 – 31 December 2026
- **Status:** Active 05\_Northstar\_Logistics\_Enterpri...

So think of it like this:



This document describes the **special terms Northstar negotiated with ParcelPilot**.

## 2. Why do we need this document?

Remember the general Support Policy v3?

For a normal Enterprise customer:

P1 → 30 minutes  
P2 → 2 hours  
P3 → 1 business day

01\_Support\_Policy\_v3\_CURRENT

But Northstar's contract gives them **different response targets**.

Therefore, if our AI sees:

"Northstar has a P1 incident."

it cannot simply look at the general Enterprise rule.

It must check:

**"Does Northstar have a special agreement?"**

Yes.

Then the agreement takes priority.

## 3. Northstar's support response times

The Northstar agreement says:

| Severity | Northstar response target |
|----------|---------------------------|
| P1       | 15 minutes, 24×7          |
| P2       | 1 hour                    |
| P3       | 8 business hours          |

05\_Northstar\_Logistics\_Enterpri...

Compare that with the normal Enterprise policy:

## Severity

## General Enterprise

P1

30 minutes

P2

2 hours

P3

1 business day

01\_Support\_Policy\_v3\_CURRENT

So Northstar gets **faster support**.

## 4. Let's understand P1

Suppose Northstar reports:

"Our entire shipment system is down."

We determine:

Problem

↓

Critical

↓

P1

General Enterprise policy would say:

Respond within 30 minutes.

But Northstar's agreement says:

Respond within **15 minutes, 24×7**.

05\_Northstar\_Logistics\_Enterpri...

So our AI must use:

**15 minutes**

not:

**30 minutes.**

## 5. Why does Northstar get a different time?

Because this is a **customer-specific contract**.

Think about a normal service:

Standard terms

↓

Everyone gets default service

But a large customer might negotiate:

"We need faster support because our business depends heavily on your platform."

So the company signs a special agreement.

That agreement becomes the customer's specific set of rules.

The Support Policy explicitly establishes that signed customer agreements have priority over the general current Support Policy. 01\_Support\_Policy\_v3\_CURRENT

---

## 6. The most important part: cancellation

This is probably the **most important rule in this PDF for our project**.

The general Cancellation SOP says:

BOOKED

+

within 30 minutes

→ ₹0

BOOKED

+

more than 30 minutes

→ ₹250 normally

03\_Cancellation\_and\_Service\_Cre...

But Northstar's agreement says:

Northstar can cancel **any BOOKED shipment before pickup without a cancellation fee**, regardless of how long ago it was booked. 05\_Northstar\_Logistics\_Enterpri...

This is a direct example of a customer-specific override.

---

## 7. Let's use an example



Imagine:

```
Order:
ORD-1001

Customer:
Northstar

Status:
BOOKED

Booked:
2 hours ago
```

The general SOP says:

```
BOOKED
+
>30 minutes
↓
₹250 fee
```

But then we check:

```
Customer = Northstar
↓
Northstar Agreement exists
↓
Agreement says:
BOOKED before pickup = ₹0 cancellation fee
```

Therefore:

**Final answer:**

**Northstar can cancel it without a cancellation fee.**

---

## 8. Why this is important for our AI

This single example shows why our agent cannot simply do:

```
Question
↓
Search
↓
First relevant answer
```

Because the system could retrieve:

Cancellation SOP:  
₹250

and stop there.

That would be wrong.

It needs to do:

```
Find order
 ↓
Identify customer
 ↓
Find customer's agreement
 ↓
Find general rule
 ↓
Compare
 ↓
Customer agreement overrides
 ↓
Answer
```

This is **source precedence + customer context**.

---

## 9. Northstar's service-credit rule

The agreement also specifies a:

**Monthly aggregate service-credit cap of ₹5,000.**

05\_Northstar\_Logistics\_Enterpri...

"Monthly aggregate" simply means:

**There is a total limit of ₹5,000 in service credits for Northstar during a month.**

For example, conceptually:

```
January

Credit 1 → ₹1,000
Credit 2 → ₹2,000
Credit 3 → ₹2,000

Total → ₹5,000
```

Once the monthly cap is reached, the system needs to consider that agreement limit before granting additional credits.

**Important:** the document says the current service-credit SOP otherwise applies.

05\_Northstar\_Logistics\_Enterpri...

So we shouldn't interpret the ₹5,000 cap as replacing every other service-credit rule.

---

## 10. What does "otherwise the SOP applies" mean?

This is important.

Northstar's agreement doesn't create an entirely separate support system.

Instead:

```
General SOP
 ↓
Default rules

Northstar Agreement
 ↓
Changes specific rules
```

For example:

### General SOP

Failed pickup → default service-credit rules.

### Northstar Agreement

Monthly service-credit cap = ₹5,000.

So we use:

```
Northstar-specific rule
+
General SOP for everything else
```

This is a **partial override**, not a complete replacement.

---

## 11. Northstar's contact

The agreement identifies:

**Dedicated CSM: Priya Mehta**

05\_Northstar\_Logistics\_Enterpri...

CSM means **Customer Success Manager**.

In simple terms, this is the person responsible for helping manage the relationship between ParcelPilot and Northstar.

This information could be useful if the AI needs to recommend or route something to the customer's dedicated contact.

---

## 12. What the AI should extract from this PDF

For our project, I would turn the important information into something conceptually like:

```
Customer:
Northstar Logistics

Account:
ACCT-001

Plan:
Enterprise

Agreement:
ACTIVE

Contract:
01 Jan 2026 - 31 Dec 2026

P1:
15 min, 24x7

P2:
1 hour

P3:
8 business hours

Cancellation:
BOOKED before pickup → ₹0 fee

Service credit:
Monthly cap = ₹5,000
```

Those are the pieces our agent needs when handling Northstar questions.

05\_Northstar\_Logistics\_Enterpri...

## 13. How it connects to the other PDFs

This is where you should start seeing the architecture.

### Support Policy v3

General rules

### Cancellation SOP

General cancellation/service-credit rules

### Northstar Agreement

Northstar-specific overrides

### Excel

Actual Northstar orders/tickets/accounts

So a question like:

**"Can Northstar cancel ORD-1001 without a fee?"**

requires multiple sources.

```
graph TD
 USER --> Q["\"Cancel ORD-1001?\""]
 Q --> E["Excel lookup"]
 E --> N["ORD-1001 = Northstar"]
 N --> S["Status = BOOKED"]
 S --> C["Cancellation SOP"]
 C --> R["Normal rule = ₹250"]
 R --> A["Northstar Agreement"]
```

↓  
Special rule = ₹0  
↓  
FINAL  
₹0 fee

## 14. Another example: P1 incident

User asks:

"How quickly should ParcelPilot respond to Northstar's P1?"

The AI needs:

### Support Policy

Normal Enterprise:

**30 minutes.** 01\_Support\_Policy\_v3\_CURRENT

### Northstar Agreement

Northstar:

**15 minutes, 24×7.** 05\_Northstar\_Logistics\_Enterpri...

Then:

Customer = Northstar  
↓  
Agreement exists  
↓  
Agreement has higher priority  
↓  
P1 = 15 minutes

## 15. What you should NOT assume

There are some things this document **doesn't tell us**.

For example, don't assume:

"Northstar has special rules for everything."

No.

The agreement specifically gives us certain overrides.

For cancellation, it gives a special waiver.

For service credits, it gives a monthly cap.

For support response times, it gives different targets.

For other matters, we should continue using the applicable general policies unless the agreement says otherwise.

That distinction prevents us from over-applying customer-specific rules.

---

## 16. The most important lesson from this PDF

This document teaches us:

**A customer can have rules that are different from the company's default rules.**

Therefore, our AI must know:

```
Who is the customer?
 ↓
Does the customer have an agreement?
 ↓
What does the agreement change?
 ↓
What general rules still apply?
 ↓
Make the decision
```



## Clear notes for your notebook

If you want a short version to study, write this:

```
05_Northstar_Logistics_Enterprise_Agreement

Customer:
Northstar Logistics

Account:
ACCT-001

Plan:
Enterprise
```

Agreement:

Active

1 Jan 2026 - 31 Dec 2026

SUPPORT:

P1 → 15 min, 24×7

P2 → 1 hour

P3 → 8 business hours

CANCELLATION:

Any BOOKED shipment before pickup

→ Can cancel

→ ₹0 fee

→ No matter how long ago it was booked

SERVICE CREDIT:

Monthly aggregate cap

→ ₹5,000

CONTACT:

Dedicated CSM → Priya Mehta

IMPORTANT:

Customer agreement overrides general policy  
where it provides a different rule.

05\_Northstar\_Logistics\_Enterpri...

## ★ One sentence to remember

**Northstar is an Enterprise customer with a special active contract, so when Northstar's contract gives a different rule from ParcelPilot's general policy, the Northstar-specific rule applies.**

That is the **main purpose of this PDF** in our AI-agent project.

Sources

Sources

. 06\_LumenWorks\_Service\_Agreement.pdf explain about  
this

Absolutely. Let's understand 06\_LumenWorks\_Service\_Agreement.pdf from zero, just like we did with Northstar.



The most important idea is:

**LumenWorks is another ParcelPilot customer, but its contract has different special rules from Northstar and from the general ParcelPilot policies.**

So this document teaches our AI:

**"Don't assume every customer has the same rules."**

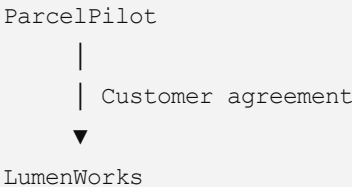
## 1. Who is LumenWorks?

The agreement identifies:

- **Customer:** LumenWorks
- **Account:** ACCT-002
- **Plan:** Growth
- **Agreement:** Active
- **Contract period:** 1 March 2026 – 28 February 2027

06\_LumenWorks\_Service\_Agreement

So conceptually:



This is a **customer-specific contract**, just like the Northstar agreement.

## 2. Why do we need this agreement?

Because the general ParcelPilot policy says one thing, but **LumenWorks has negotiated different terms for some areas.**

For example, the normal Growth support targets are:

| Severity | General Growth   |
|----------|------------------|
| P1       | 2 business hours |
| P2       | 4 business hours |

## Severity

## General Growth

P3

2 business days

01\_Support\_Policy\_v3\_CURRENT

LumenWorks' agreement gives:

## Severity

## LumenWorks

P1

2 business hours

P2

4 business hours

P3

2 business days

So in this case, **the response targets happen to match the general Growth policy.**

06\_LumenWorks\_Service\_Agreement

But there is an important difference:

**LumenWorks has no weekend or after-hours support coverage.**

06\_LumenWorks\_Service\_Agreement

So we shouldn't assume that because it is a P1, support is available 24×7.

## 3. Let's understand "no weekend or after-hours coverage"

This is important.

Imagine LumenWorks reports a P1 issue at:

Saturday, 11 PM.

The general Growth response target says:

P1 = 2 business hours.

But LumenWorks' agreement says:

**No weekend or after-hours support coverage.**

06\_LumenWorks\_Service\_Agreement

So the AI **cannot simply treat LumenWorks like a 24×7 customer.**

This is exactly why the customer agreement needs to be checked.

---

## 4. Cancellation rules

Now let's look at cancellation.

The LumenWorks agreement says:

**There is no special cancellation-fee waiver.**

06\_LumenWorks\_Service\_Agreement

This means LumenWorks does **not** get a special cancellation discount like Northstar.

Remember Northstar:

Northstar  
BOOKED before pickup  
→ ₹0 cancellation fee

05\_Northstar\_Logistics\_Enterpri...

LumenWorks:

No special waiver  
↓  
Use general Cancellation SOP

So if LumenWorks has:

BOOKED  
+  
>30 minutes

the normal SOP applies:

**₹250 cancellation fee**, unless another applicable rule changes the result.

03\_Cancellation\_and\_Service\_Cre...

---

## 5. The BIG special rule: service credits

This is the most important part of the LumenWorks agreement.

The general service-credit SOP says:

```
Pickup >2 hours late
+
Carrier at fault
+
Customer not at fault
↓
Lower of ₹500 or 10% of shipment fee
```

03\_Cancellation\_and\_Service\_Cre...

But LumenWorks has a different rule.

---

## 6. LumenWorks' special service-credit rule

For LumenWorks:

```
Pickup >4 hours late
+
Carrier at fault
+
Customer not at fault
↓
₹300 fixed credit
```

06\_LumenWorks\_Service\_Agreement

The agreement explicitly says this **replaces the default threshold and credit amount**.

This is important.

It isn't simply adding another condition.

It is **overriding the general rule**.

---

## 7. Let's compare the two rules

### General ParcelPilot SOP

```
>2 hours late
+
Carrier at fault
+
Customer not at fault
```

## LumenWorks

```
>4 hours late
+
Carrier at fault
+
Customer not at fault
↓
₹300 fixed
```

So:

| Situation       | General SOP             | LumenWorks         |
|-----------------|-------------------------|--------------------|
| Delay threshold | >2 hours                | <b>&gt;4 hours</b> |
| Carrier fault   | Required                | Required           |
| Customer fault  | Must be absent          | Must be absent     |
| Credit          | Lower of ₹500 / 10% fee | <b>₹300 fixed</b>  |

03\_Cancellation\_and\_Service\_Cre...

06\_LumenWorks\_Service\_Agreement

## 8. Example: 3-hour delay

Suppose:

```
Customer = LumenWorks
Pickup delay = 3 hours
Carrier = at fault
Customer = not at fault
```

Someone who only knows the general SOP might say:

"The customer qualifies because the pickup is more than 2 hours late."

That would be **wrong for LumenWorks**.

Why?

Because LumenWorks has a special threshold:

**More than 4 hours.**

3 hours is not more than 4 hours.

Therefore:

**No service credit under the LumenWorks agreement.**

---

## 9. Example: 5-hour delay

Now:

```
Customer = LumenWorks
Pickup delay = 5 hours
Carrier = at fault
Customer = not at fault
```

Check the LumenWorks rule:

```
5 hours > 4 hours
 ✓
Carrier at fault
 ✓
Customer not at fault
 ✓
```

Therefore:

**₹300 service credit.**

Not:

₹500

Not:

10% of shipment fee

Because LumenWorks' agreement specifically replaces the default credit amount with

**₹300 fixed.** 06\_LumenWorks\_Service\_Agreement

---

## 10. What if the shipment fee is ₹10,000?

Under the general rule:

10% of ₹10,000 = ₹1,000

Then the lower of:

₹500

₹1,000

would be:

**₹500**

But for LumenWorks:

**₹300 fixed.**

So:

General SOP → ₹500

LumenWorks → ₹300

The LumenWorks rule wins because it is the customer-specific agreement.

---

## 11. What if the carrier isn't at fault?

Suppose:

Pickup delay = 5 hours

Carrier fault = NO

Even though:

5 > 4

the customer does **not** satisfy the complete LumenWorks rule.

The agreement requires:

- more than 4 hours
- carrier at fault
- customer not at fault.

06\_LumenWorks\_Service\_Agreement

So:

**No credit based on this rule.**

## 12. What if we don't know who is responsible?

Suppose the system says:

"Pickup was delayed by 5 hours."

But we don't know:

Carrier fault = ?  
Customer fault = ?

We shouldn't tell LumenWorks:

"You definitely get ₹300."

The general SOP says not to promise a service credit when important facts such as carrier fault or customer fault are unknown. 03\_Cancellation\_and\_Service\_Cre...

So the agent should say something like:

"The pickup appears to have exceeded the LumenWorks delay threshold, but I need to verify responsibility before confirming the credit."

That's safer.

## 13. The most important comparison: Northstar vs LumenWorks

This is where the assessment becomes very interesting.

We have two customers:

### Northstar

Enterprise  
P1 → 15 min  
P2 → 1 hour  
P3 → 8 business hours  
  
BOOKED cancellation  
→ ₹0  
  
Service credit  
→ ₹5,000 monthly aggregate cap



## LumenWorks

Growth

P1 → 2 business hours

P2 → 4 business hours

P3 → 2 business days

No weekend/after-hours coverage

No special cancellation waiver

Service credit:

>4 hours

+

carrier fault

+

customer not at fault

→ ₹300

So the AI must **identify the customer first**.

---

## 14. This is why customer identification matters

Suppose the user asks:

"The pickup was 3 hours late. Do we give a credit?"

We cannot answer yet.

We need:

Which customer?

↓

Northstar?

LumenWorks?

Other?

Because the answer may be different.

For LumenWorks:

3 hours late

↓

Below 4-hour threshold

↓

No credit under special agreement

For another customer following the default SOP:

3 hours late

↓

Above 2-hour threshold

↓

Could qualify, assuming other conditions are satisfied

So:

**Same operational event → potentially different answer depending on customer.**

This is one of the central challenges of the whole project.

---

## 15. What should we extract from this PDF?

For our AI system, the important information is:

Customer:

LumenWorks

Account:

ACCT-002

Plan:

Growth

Agreement:

ACTIVE

Contract:

1 Mar 2026 - 28 Feb 2027

Support:

P1 → 2 business hours

P2 → 4 business hours

P3 → 2 business days

Coverage:

No weekend / after-hours support

Cancellation:

No special fee waiver

Service Credit:

```
>4 hours late
+ carrier at fault
+ customer not at fault
→ ₹300 fixed
```

06\_LumenWorks\_Service\_Agreement

## 16. How this connects to our AI architecture

This PDF makes the workflow clearer.

Suppose the user asks:

**"Does LumenWorks qualify for a pickup credit?"**

Our agent should do:

```
USER
↓
Identify customer
↓
LumenWorks
↓
Get order/pickup data from Excel
↓
Check actual delay
↓
Check carrier responsibility
↓
Check customer responsibility
↓
Retrieve LumenWorks Agreement
↓
Apply >4 hour rule
↓
Calculate ₹300
↓
Answer
```

So the AI isn't just doing:

Question → PDF → Answer

It's doing:

```
Question
↓
Customer context
```

↓  
Operational data  
↓  
Applicable agreement  
↓  
General SOP  
↓  
Decision



## Clear notes for your notebook

Write this:

06\_LumenWorks\_Service\_Agreement

Customer:

LumenWorks

Account:

ACCT-002

Plan:

Growth

Agreement:

Active

1 Mar 2026 - 28 Feb 2027

SUPPORT:

P1 → 2 business hours

P2 → 4 business hours

P3 → 2 business days

COVERAGE:

No weekend / after-hours support.

CANCELLATION:

No special cancellation-fee waiver.

→ Use general Cancellation SOP.

SERVICE CREDIT:

>4 hours late

+

Carrier at fault

+

Customer not at fault

→ ₹300 fixed credit.

IMPORTANT:

LumenWorks agreement overrides the default service-credit threshold and amount.

06\_LumenWorks\_Service\_Agreement

---

## ★ The one thing to remember

**Northstar and LumenWorks are deliberately different.**

That's not accidental.

CalQuity wants us to build an AI that doesn't think:

"I know the general ParcelPilot rule, so I'll apply it to everyone."

Instead, it should think:

**"Who is this customer? What agreement applies to them? What does the general policy say? Does the customer's agreement override it? What does the actual operational data show?"**

That is the core reasoning pattern we're going to build into the agent.

Sources

Sources

ChatGPT can make mistakes. Check important info.